

Informatik 2 Zusammenfassung

von Loris Hliddal

1. Python-Syntax — Übersicht

1.1. Datentypen

```
x = 5 #int
y = 3.14 #float
z = "hello" #string
b = True #boolean
n = None #null / leer
c = 3 + 2j #komplexe Zahl
type(x) #gibt <class 'int'> zurück
```

1.2. Kontrollstrukturen (if / else / while / for)

```
# von 0 bis 10 in 2er-Schritten
for i in range(0, 10, 2):
    print(i)

for char in "abc": # über Zeichen iterieren
    print(char)

# Index + Wert
for i, val in enumerate(["a", "b"]):
    print(i, val)

# Dict iterieren
for k, v in {"x": 1, "y": 2}.items():
    print(k, v)
```

1.3. Funktionen

```
# Funktionsdefinition
def add(a, b):
    return a + b

# Aufruf mit Argumenten
add(2, 3)

def default_arg(x=1): # Standardparameter
    return x + 1

# variable Positionsargumente
def varargs(*args):
    return args

# variable Schlüsselwortargumente
def kwargs(**kw):
    return kw

double = lambda x: x*2 # anonyme Funktion
```

1.4. Python-Container

```
# list: geordnet, veränderbar
lst = [1, 2, 3]
# tuple: geordnet, unveränderbar
tup = (1, 2, 3)
# set: ungeordnet, einzigartig
s = {1, 2, 3}

# dict: Schlüssel-Wert-Paare
d = {"a": 1, "b": 2}
# string: geordnet, unveränderbar
s = "abc"
```

1.5. Operationen auf Containern

```
len(lst) #Anzahl Elemente
1 in lst #Enthaltensein prüfen
lst[0] #Indexzugriff
lst[1:3] #Slicing
lst[start:stop:step] #Slicing mit Schrittweite
max(lst), min(lst) #Maximum / Minimum
sum(lst) #Summe aller Elemente
list("abc") #in list umwandeln
tuple(lst) #in tuple umwandeln
set(lst) #in set umwandeln
sorted(lst) #sortierte Kopie
```

1.6. Sequenzen (geordnete Container)

```
# veränderbare Sequenz
lst = [10, 20, 30]
# unveränderbare Sequenz
tup = (10, 20, 30)
# unveränderbare Zeichenkette
s = "abc"
lst[0], tup[1], s[2] # Indexzugriff
lst[1:] # Slicing
```

1.7. Allgemeine Sequenz-Operationen

```
[1, 2] + [3, 4] # Verkettung
[1] * 3 # Wiederholung
# Liste fester Länge mit Standardwert
[0] * M
# verschachtelte Listen: unabhängige innere Listen erzeugen
[[ for _ in range(M)] # empfohlen
[[]] * M # dieselbe innere Liste wiederholt
x = [1, 2, 3]
x.index(2) # Index von 2 suchen
x.count(1) # Anzahl der 1en zählen
# über Sequenz iterieren
for item in x:
    print(item)
```

1.8. Slicing — Details

```
a = [10,20,30,40,50] # a[start:stop:step]
# start inklusive, stop exklusiv, Standardschritt=1
a[1:4] # [20,30,40] a[:3] / a[2:] / a[:]=copy
a[-1] # 50 a[-2:] = last 2, a[:-2] = drop 2
a[:2] # [10,30,50] a[::-1] = reversed copy
a[4:1:-1] # [50,40,30] step<0 →right to left
b = a[:]; b[0]=99 # slice = new list (no alias)
a[1:3] = [0,0,0] # slice-assign replaces range
a[1:3] = [] # slice-assign [] deletes
"python"[:-1] # works on str/tuple (immut.)
```

1.9. Häufige Listen-Operationen

```
lst.append(4) #ans Ende anfügen
lst.extend([5, 6]) #mehrere Elemente anfügen
lst.insert(1, 99) #an Index einfügen
lst.remove(1) #erstes Vorkommen entfernen
lst.pop() #letztes Element entfernen
del lst[0] #per Index löschen
lst.sort() #in-place sortieren
lst.reverse() #in-place umkehren
```

1.10. List Comprehension

```
[x**2 for x in range(5)] # Quadrate
[x for x in range(10) if x % 2 == 0]# gerade Zahlen
# verschachtelte Schleifen
[(i, j) for i in [1,2] for j in [3,4]]
```

1.11. Häufige String-Operationen

```
s = " Hello World "
```

--- Leerzeichen und Bereinigung ---
führende/abschliessende Leerzeichen entfernen
s.strip()
nur links/rechts Leerzeichen entfernen
s.lstrip(), s.rstrip()

1.12. String-Methoden — Referenz

```
# --- Gross-/Kleinschreibung ---  
s.lower()      # alles klein  
s.upper()      # alles gross  
# erster Buchstabe gross, Rest klein  
s.capitalize()  
# jedes Wort gross geschrieben  
s.title()  
# Gross-/Kleinschreibung tauschen: "aA" → "Aa"  
s.swapcase()
```

```
# --- Präfix/Suffix und Suche ---  
# True, wenn String mit "He" beginnt  
s.startswith("He")  
# True, wenn String mit "ld" endet  
s.endswith("ld")  
# Index der ersten Übereinstimmung oder -1  
s.find("lo")  
s.rfind("l")      # Index der letzten Übereinstimmung  
# wie find, aber Exception wenn nicht gefunden  
s.index("lo")  
# Häufigkeit des Teilstrings zählen  
s.count("l")
```

```
# --- Ersetzen und Ändern ---  
# alle Vorkommen ersetzen  
s.replace("World", "Python")  
# nur erstes Vorkommen ersetzen  
s.replace("l", "*", 1)
```

```
# --- Aufteilen und Verbinden ---  
s.split()         # nach Leerzeichen aufteilen  
# nach eigenem Trennzeichen aufteilen  
s.split("o")  
# von rechts aufteilen, max. 1 Trennung  
s.rsplit(" ", 1)  
# aufteilen in (vor, Treffer, nach)  
s.partition(" ")  
s.rpartition(" ") # dasselbe, aber von rechts  
# Liste mit Bindestrich zu String verbinden  
"_".join(["a", "b"])  
# String aus Zeichen zusammenbauen  
"".join(list("abc"))
```

```
# --- Prüfungen ---  
"abc".isalpha()   # nur Buchstaben?  
"123".isdigit()   # nur Ziffern?  
# nur 0-9? (hier False)  
"12.3".isdecimal()  
# nur Buchstaben und Zahlen?  
"abc123".isalnum()  
" ".isspace()     # nur Leerzeichen?  
"Title Case".istitle() # Titelschreibung?  
"ABC".isupper()   # alles gross?  
"abc".islower()   # alles klein?
```

```
# --- Ausrichtung / Auffüllen ---  
# links mit Nullen auffüllen → "00042"  
"42".zfill(5)  
# mittig ausrichten  
"hi".center(10)  
# links ausrichten mit Zeichen  
"hi".ljust(8, "-")  
# rechts ausrichten mit Zeichen  
"hi".rjust(8, ".")
```

```
# --- Sonstiges ---  
# in Zeichenliste umwandeln  
list("abc")  
# Ganzzahlen zu String verbinden  
", ".join(map(str, [1, 2, 3]))  
s[::-1]           # String umkehren
```

1.13. Collections (ungeordnete Container)

```
# set: keine Duplikate  
s = {1, 2, 3}
```

d = {"x": 1, "y": 2} # dict: Schlüssel-Wert

sets sind schnell für Enthaltensein-Prüfung,
kein Indexzugriff
2 in s # Enthaltensein prüfen
len(s) # Anzahl Elemente

1.14. Set-Operationen

```
a = {1, 2, 3}, b = {3, 4, 5}  
a | b # Vereinigung  
a & b # Schnittmenge  
a - b # Differenz  
a ^ b # symmetrische Differenz  
a.add(6) # Element hinzufügen  
a.remove(1) # Element entfernen  
x in a # True, wenn x in a enthalten
```

1.15. Dictionary-Operationen

```
d = {"a": 1, "b": 2}  
d["c"] = 3 # Schlüssel hinzufügen / aktualisieren  
d.get("a", 0) # sicherer Zugriff  
del d["b"] # Schlüssel löschen  
"a" in d # Enthaltensein prüfen  
list(d.keys()) # Liste der Schlüssel  
list(d.values()) # Liste der Werte  
list(d.items()) # Liste der (Schlüssel, Wert)-Paare
```

1.16. Iteration über ein Dictionary

```
d = {"x": 1, "y": 2}
```

for k in d: # Schlüssel
 print(k)

for v in d.values(): # Werte
 print(v)

for k, v in d.items(): # Schlüssel und Werte
 print(k, v)

mit Index
for i, (k, v) in enumerate(d.items()):
 print(i, k, v)

1.17. Dictionary- / Set-Comprehension

```
# Dict Comprehension
{x: x**2 for x in range(5)}
# gefiltertes Dict
{k: v for k, v in d.items() if v > 1}
# Set Comprehension
{x for x in range(10) if x % 2 == 0}
```

1.18. zip()

```
a = [1, 2, 3]
b = ['a', 'b', 'c']

# Elemente paaren: (1,'a'), (2,'b'), ...
zipped = zip(a, b)
# [(1, 'a'), (2, 'b'), (3, 'c')]
print(list(zipped))

# auch in for-Schleifen verwendbar
for x, y in zip(a, b):
    print(x, y)
```

1.19. zip() — Entpacken

```
# gezippte Listen entpacken
a2, b2 = zip(*[(1, 'a'), (2, 'b'), (3, 'c')])
print(a2)          # (1, 2, 3)
print(b2)          # ('a', 'b', 'c')

# zip endet beim kürzesten Input
# [(1, 'a'), (2, 'b')]
print(list(zip([1, 2], ['a', 'b', 'c'])))
```

2. Klassen

2.1. Vererbung

Oberklasse / Unterklasse

Eine **Oberklasse** (Superclass) ist eine Klasse, die als Basis für eine andere Klasse dient – die sogenannte **Unterklasse** (Subclass). Mittels `super().__init__()` kann die Unterklasse auf den Konstruktor der Oberklasse zugreifen.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def honk(self):
        return f"{self.model} is honking!"
```

Oberklasse innerhalb der Unterklasse

Python

```
class ElectricCar(Car):
    def __init__(self, brand, model):
        # ruft __init__ der Elternklasse Car auf
        super().__init__(brand, model)

    def charge(self):
        return f"{self.model} battery is charging."
```

2.2. Klasseninterne Funktionen implementieren

Achtung

Wenn Operatoren wie `+` für Klassen überschrieben werden, sollte das Ergebnis wieder eine Instanz dieser Klasse sein. Sonst handelt es sich um einen undefinierten Datentyp, auf den keine Methoden wie `__str__` angewendet werden können.

```
class CustomString:
    def __init__(self, string):
        self.string = string

    def __str__(self):
        return self.string

    def __add__(self, other):
        result_string = ""
        for i in range(len(self.string)):
            if i < len(other.string):
                result_string += (self.string[i]
                                   + other.string[i])
            else:
                result_string += self.string[i]
        return CustomString(result_string)

s1 = CustomString("abcd")
s2 = CustomString("123")
print(s1 + s2)
```

3. Dynamische Programmierung

Kernidee

Ein komplexes Problem wird in kleinere Teilprobleme zerlegt. Diese werden einmal gelöst und das Ergebnis gespeichert (in einer Liste, Tabelle o.ä.), um es später bei Bedarf wiederzuverwenden.

- **Memoization (Top-down)** — Beginnt mit dem ursprünglichen Problem und löst es rekursiv, speichert aber bereits berechnete Ergebnisse in einem Speicher (z.B. einer Liste oder einem Dictionary). Der Speicher verhindert doppelte Berechnungen.
- **Dynamische Programmierung (Bottom-up)** — Baut die Lösung iterativ von den kleinsten Teilproblemen auf. Die Ergebnisse werden schrittweise gespeichert und zur Lösung größerer Probleme verwendet. Keine Rekursion nötig.

Unterschied

Beide Ansätze nutzen Speicher zur Optimierung, aber:

- **Memoization** — rekursiv + speichert nur tatsächlich verwendete Teilprobleme.
- **DP (Bottom-up)** — iterativ + berechnet alle möglichen Teilprobleme systematisch.

3.1. [EHA] Mustererkennung: Welcher Zustand?

Zustandsraum	Muster	Beispiele
$dp[i]$	1D-Sequenz	Fibonacci, House Robber
$dp[i][w]$	Kapazität	Knapsack, Coin Change
$dp[i][j]$	Zwei-Index	Grid, LCS, Edit Dist.
$dp[i][j], i \leq j$	Intervall + Split	Matrix Chain, Opt. BST

3.2. 1D-DP: $dp[i]$ — Lineare Sequenzen

- **Zustand** — $dp[i]$ = optimale Lösung für die ersten i Elemente
- **Übergang** — $dp[i] = f(dp[i-1], dp[i-2], \dots)$
- **Basisfall** — $dp[0], dp[1]$ explizit setzen
- **Antwort** — $dp[n]$
- **Komplexität** — $O(n)$ Zeit, $O(n)$ Raum

Fibonacci — Bottom-up

Iterative Berechnung. Berechnet auch unnötige Elemente, geht in 1D.

```
def fib_DP(n): # gesucht: n-te Fibonacci-Zahl
    F = [None] * (n+1) # Tabelle anlegen: Indizes 0..n
    # Basisfaelle: F(0) = F(1) = 1
    F[0] = 1
    F[1] = 1
    # Bottom-Up: Tabelle von klein nach gross befüllen
    for i in range(2, n+1):
        F[i] = F[i-1] + F[i-2] # Rekurrenz anwenden
    return F[n] # Ergebnis: letzter Eintrag
```

Fibonacci — Top-down

Rekursive Berechnung mit Memoization. Berechnet nur benötigte Elemente, meistens in 2D. (→ 12.2)

```
def fibonacci(n, memo = None):
    # memo: Zwischenspeicher fuer bereits berechnete Werte
    if memo is None:
        memo = [1,1] + [None]*(n-1)
        if memo[n] is not None: # bereits berechnet -> zurueck
            return memo[n]
        else: # noch nicht berechnet -> rekursiv
            memo[n-1] = fibonacci(n-1, memo)
            memo[n-2] = fibonacci(n-2, memo)
            memo[n] = memo[n-1] + memo[n-2] # Rekurrenz + speichern
    return memo[n]
```

[EHA] House Robber / Coin Row

- **Problem** — Array mit Werten; keine zwei direkt benachbarten Elemente gleichzeitig wählbar. Maximiere die Summe.
- **Übergang** — $dp[i] = \max(dp[i-1], dp[i-2] + val[i])$

```
def house_robber(nums):
    n = len(nums)
    dp = [0] * n # dp[i]: max. Summe bis Index i
    dp[0] = nums[0] # Basisfall: nur erstes Element
    # Basisfall: erstes/zweites Element
    dp[1] = max(nums[0], nums[1])
    # nehmen: aktuell + bestes ohne direkten Nachbar (i-2)
    # weglassen: bestes Ergebnis bis zum Vorgaenger (i-1)
    for i in range(2, n):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i])
    return dp[-1] # Antwort: letzter Eintrag
```

3.3. [EHA] Kapazitäts-DP: $dp[i][w]$ — Knapsack-Familie

- **Zustand** — $dp[i][w]$ = opt. Wert mit ersten i Objekten bei Kapazität w
- **Übergang** — Objekt nehmen oder weglassen
- **Basisfall** — $dp[0][*] = dp[*][0] = 0$
- **Antwort** — $dp[n][W]$
- **Komplexität** — $O(n \cdot W)$ Zeit und Raum
- **Varianten:**
 - **0/1 Knapsack** — jedes Objekt max. einmal wählbar
 - **Unbounded** (Coin Change, Rod Cutting) — Objekte wiederholbar
 - **Subset Sum** — Entscheidungsproblem (bool)

[EHA] 0/1 Knapsack

- **Übergang** — $dp[i][w] = \max(dp[i-1][w], val[i] + dp[i-1][w-wt[i]])$

```
def knapsack(wt, val, W):
    n = len(val)
    # dp[i][w]: max. Wert mit ersten i Objekten, Kapazitaet w
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1): # ueber alle Objekte iterieren
        for w in range(W+1):
            dp[i][w] = dp[i-1][w] # Option 1: weglassen
            if wt[i-1] <= w: # Option 2: nehmen (passt?)
                dp[i][w] = max(dp[i-1][w],
                               val[i-1] + dp[i-1][w-wt[i-1]])
    return dp[n][W] # Antwort: rechte untere Ecke
```

[EHA] Coin Change (Unbounded)

- **Übergang** — $dp[a] = \min(dp[a], dp[a-c] + 1)$ für alle Münzen c

```
def coin_change(coins, amount):
    # dp[a]: minimale Muenzanzahl fuer Betrag a
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0 # Basisfall: 0 Muenzen fuer Betrag 0
    for a in range(1, amount + 1): # Betrag aufbauen
        for c in coins:
            if c <= a: # Muenze c passt noch in Betrag a
                # 1 Muenze c + optimale
                # Loesung fuer Restbetrag
                dp[a] = min(dp[a], dp[a-c] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

3.4. [EHA] Zwei-Index-DP: $dp[i][j]$ — Gitter & Strings

- **Zustand** — $dp[i][j]$ = opt. Lösung für Präfix i von A und Präfix j von B
- **Übergang** — aus $dp[i-1][j-1], dp[i-1][j], dp[i][j-1]$
- **Basisfall** — erste Zeile & Spalte (Index 0) explizit setzen
- **Antwort** — $dp[m][n]$
- **Komplexität** — $O(m \cdot n)$ Zeit und Raum

[EHA] Longest Common Subsequence (LCS)

Übergang:

- **X[i]==Y[j]** — $dp[i][j] = dp[i-1][j-1] + 1$
- **sonst** — $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

```
def lcs(X, Y):
    m, n = len(X), len(Y)
    # dp[i][j]: Laenge der LCS von X[0..i-1] und Y[0..j-1]
    # Basisfall: dp[0][*] = dp[*][0] = 0 (leere Teilfolge)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]: # Zeichen gleich: diagonal +1
                dp[i][j] = dp[i-1][j-1] + 1
            else: # ungleich: bestes von oben/links
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n] # Antwort: rechte untere Ecke
```

[EHA] Grid: Minimale Pfadkosten

- **Übergang** — $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$

```
def min_path(grid):
    m, n = len(grid), len(grid[0])
    # dp[i][j]: minimale Kosten von (0,0) bis Zelle (i,j)
    dp = [[0]*n for _ in range(m)]
    dp[0][0] = grid[0][0] # Startpunkt
    for j in range(1, n): # erste Zeile: nur von links
        dp[0][j] = dp[0][j-1] + grid[0][j]
    for i in range(1, m): # erste Spalte: nur von oben
        dp[i][0] = dp[i-1][0] + grid[i][0]
    for i in range(1, m):
        for j in range(1, n):
            # Zellkosten + bester Vorgaenger (oben/links)
            dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])

    return dp[m-1][n-1] # Antwort: rechte untere Ecke
```

3.5. [EHA] Intervall-DP: $dp[i][j]$ — Split-Index k

- **Zustand** — $dp[i][j]$ = optimale Lösung für Teilproblem über $[i..j]$
- **Übergang** — probe alle Split-Punkte $k \in [i, j-1]$: $dp[i][j] = \min_k \{dp[i][k] + dp[k+1][j] + cost(i, j)\}$
- **Basisfall** — $dp[i][i] = 0$ (Intervall der Länge 1)
- **Antwort** — $dp[0][n-1]$
- **Komplexität** — $O(n^3)$ Zeit, $O(n^2)$ Raum
- **Füllreihenfolge** — äussere Schleife über Länge ℓ , nicht über Zeilen!

[EHA] Matrix Chain Multiplication

Gegeben n Matrizen, Dimensionen via Array p (Länge $n+1$).

- **Übergang** — $dp[i][j] = \min_k \{dp[i][k] + dp[k+1][j] + p[i] \cdot p[k+1] \cdot p[j+1]\}$

```
def matrix_chain(p):
    n = len(p) - 1 # Anzahl Matrizen
    # dp[i][j]: min. Mult. fuer Matrizen i bis j
    dp = [[0]*n for _ in range(n)]
    for l in range(2, n+1): # Intervall-Laenge aufsteigend
        for i in range(n - l + 1):
            for j in range(i + l, n):
                j = i + l - 1 # rechtes Ende des Intervalls
                dp[i][j] = float('inf')
                # alle Split-Punkte probieren
                for k in range(i, j):
                    # linke Gruppe + rechte Gruppe
                    # + Kosten Kombination
                    cost = (dp[i][k] + dp[k+1][j] +
                        p[i] * p[k+1] * p[j+1])
                    dp[i][j] = min(dp[i][j], cost)
    return dp[0][n-1] # Antwort: gesamtes Intervall
```

4. Algorithmen

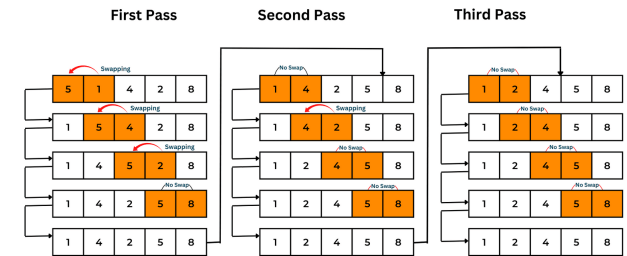
	# swaps			# comparisons			
sort	case	A	runtime	case	A	runtime	general
Bubble	worst	reverse sorted	$\Theta(n^2)$	all	-	$\Theta(n^2)$	$\Theta(n^2)$
	average	-	$\Theta(n^2)$				
	best	sorted	$\Theta(1)$ (0)				
Selection	worst	reverse sorted	$\Theta(n)$	all	-	$\Theta(n^2)$	$\Theta(n^2)$
	average	-	$\Theta(n)$				$\Theta(n^2)$
	best	sorted	$\Theta(1)$ (0)				$\Theta(n)$
Insertion	worst	reverse sorted	$\Theta(n^2)$	worst	reverse sorted	$\Theta(n^2)$	$\Theta(n^2)$
	average	-	$\Theta(n^2)$	average	-	-	$\Theta(n^2)$
	best	sorted	$\Theta(1)$ (0)	best	sorted	$\Theta(n)$	$\Theta(n)$
Merge	all	-	$\Theta(n \log(n))$	all	-	$\Theta(n \log(n))$	$\Theta(n)$
	Braucht zusätzlich $\Theta(n)$ Speicher für die Subarrays						
Quick	worst	*	$\Theta(n \log(n))$	worst	*	$\Theta(n^2)$	$\Theta(n^2)$
	average	**	$\Theta(n \log(n))$	average	**	$\Theta(n \log(n))$	$\Theta(n \log(n))$
	best	***	$\Theta(n)$	best	***	$\Theta(n \log(n))$	$\Theta(n \log(n))$
Heap	all	-	$\Theta(n \log(n))$	all	-	$\Theta(n \log(n))$	$\Theta(n \log(n))$

4.1. Bubble Sort

- Vergleicht benachbarte Elemente und vertauscht sie bei falscher Reihenfolge.
- Wiederholt dies, bis keine Vertauschungen mehr nötig sind.
- $a[:-(i+1)]$ enthält die $i+1$ größten Elemente von a in sort. Reihe.
- **Laufzeit** — $\Theta(n^2)$ im Durchschnitt und schlechtesten Fall.

```
def bubble_sort(a):
    n = len(a)
    # Grosse Schleife ueber alle Elemente
    for i in range(n):
        # Vergleicht bis zum unsortierten Rest
        for j in range(0, n-i-1):
            # Wenn linkes Element groesser...
            if a[j] > a[j+1]:
                # ...vertausche beide
                a[j], a[j+1] = a[j+1], a[j]
```

BUBBLE SORTING

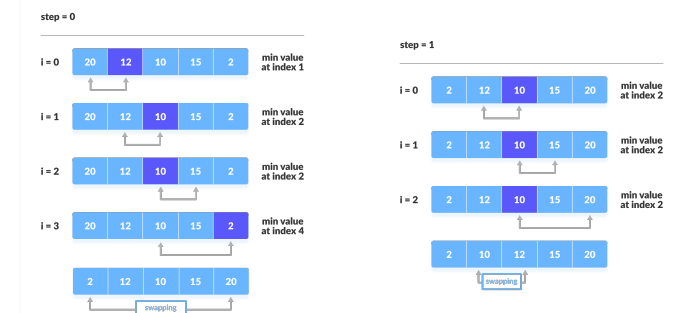


4.2. Selection Sort

Sucht in jedem Durchlauf das kleinste Element:

- Tauscht es mit dem aktuellen Element.
- Wiederholt das für den restlichen unsortierten Bereich.
- $a[:i+1]$ enthält die kleinsten $i+1$ Elemente von a , in sort. Reihe.
- **Laufzeit** — $\Theta(n^2)$ in allen Fällen.

```
def selection_sort(a):
    n = len(a)
    # Grosse Schleife ueber alle Positionen
    for i in range(n):
        # Nimm an, dass a[i] das Minimum ist
        min_idx = i
        # Suche im restlichen Feld...
        for j in range(i+1, n):
            # ...nach einem noch kleineren Element
            if a[j] < a[min_idx]:
                # Speichere neuen Index des Minimums
                min_idx = j
        # Tausche an sortierte Position
        a[i], a[min_idx] = a[min_idx], a[i]
```

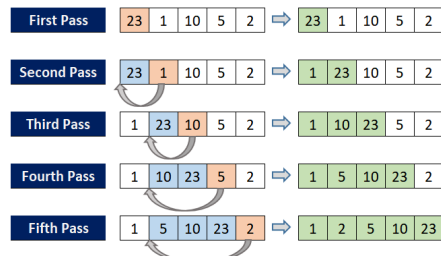


4.3. Insertion Sort

Sortiert ähnlich wie beim Kartenspiel:

- Jedes Element wird an die richtige Position im linken, bereits sortierten Teil eingefügt.
- Größere Elemente werden nach rechts verschoben.
- Die Elemente in $a[:i + 1]$ sind sortiert und entsprechen den ersten i Elementen des ursprüngl. Arrays.
- **Laufzeit** — $\Theta(n)$ im besten Fall (wenn sortiert), $\Theta(n^2)$ im Worst-Case.

```
def insertion_sort(a):
    # Starte beim zweiten Element
    for i in range(1, len(a)):
        # Speichere aktuelles Element
        key = a[i]
        # Betrachte die Elemente links davon
        j = i - 1
        # Solange links grössere Elem. sind
        while j >= 0 and key < a[j]:
            # Verschiebe sie nach rechts
            a[j + 1] = a[j]
            # Gehe weiter nach links
            j -= 1
        # Füge das Element an Lücke ein
        a[j + 1] = key
```

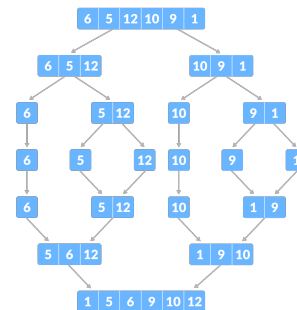


4.4. Merge Sort

Ein rekursiver **Divide-and-Conquer**-Algorithmus:

- Das Array wird in zwei Hälften geteilt.
- Beide Hälften werden rekursiv sortiert.
- Die sortierten Hälften werden zusammengeführt (merge).
- Das Resultat-Array $b[:i + 1]$ enthält die $i + 1$ kleinsten Elemente von a in sortierter Reihenfolge.
- **Laufzeit** — $\Theta(n \log n)$ in allen Fällen.

```
def merge_sort(a):
    # Wenn Array mehr als 1 Element hat
    if len(a) > 1:
        # Finde die Mitte
        mid = len(a) // 2
        # Teile das Array in zwei Hälften
        L, R = a[:mid], a[mid:]
        # Sortiere beide rekursiv
        merge_sort(L); merge_sort(R)
        i = j = k = 0
        # Beide Hälften zusammenfügen
        while i < len(L) and j < len(R):
            # Nimm das kleinere Element ...
            if L[i] < R[j]:
                # ...aus linker Hälfte
                a[k] = L[i]; i+=1
            else:
                # ...aus rechter Hälfte
                a[k] = R[j]; j+=1
            k+=1
        # Rest links anhängen
        while i < len(L):
            a[k] = L[i]; i+=1; k+=1
        # Rest rechts anhängen
        while j < len(R):
            a[k] = R[j]; j+=1; k+=1
```

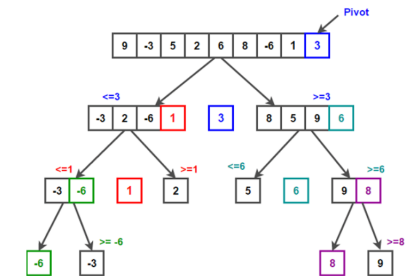


4.5. Quick Sort

Ein effizienter rekursiver **Divide-and-Conquer**-Algorithmus:

- Wählt ein *Pivot*-Element.
- Teilt das Array in Elemente kleiner und größer als das Pivot.
- Sortiert beide Teilarrays rekursiv.
- Das Pivotelement befindet sich an der richtigen Posit., Elem. links davon sind kleiner und rechts davon grösser als das Pivot..
- **Laufzeit** — $\Theta(n \log n)$ durchschnittlich, $\Theta(n^2)$ im Worst-Case.

```
def quick_sort(a, low, high):
    # Wenn mehr als ein Element da ist
    if low < high:
        # Wähle ganz rechts als Pivot
        pivot = a[high]
        # Zeiger fuer kleineres Element
        i = low - 1
        # Gehe durch das Array
        for j in range(low, high):
            # Wenn Element kleiner/gleich Pivot
            if a[j] <= pivot:
                # Verschiebe Zeiger
                i += 1
                # Vertausche Elemente
                a[i], a[j] = a[j], a[i]
        # Setze Pivot in die Mitte
        a[i+1], a[high] = a[high], a[i+1]
        # Dies ist die fertige Pivot-Position
        pi = i + 1
        # Sortiere rekursiv links vom Pivot
        quick_sort(a, low, pi - 1)
        # Sortiere rekursiv rechts vom Pivot
        quick_sort(a, pi + 1, high)
```

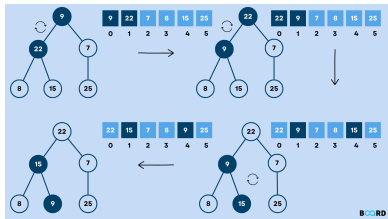


4.6. Heap Sort

- Erst wird ein Max-Heap aufgebaut. (→ 9.3)
- Grösstes Element (Wurzel) wird entfernt und ans Ende gesetzt.
- Der Heap wird danach angepasst (heapify).
- **Laufzeit** — $\Theta(n \log n)$ in allen Fällen.

```
def heapify(a, n, i):
    # Setze root als groesstes Element
    largest = i
    # Linkes und rechtes Kind
    l, r = 2 * i + 1, 2 * i + 2
    # Kind links groesser?
    if l < n and a[l] > a[largest]: largest = l
    # Kind rechts groesser?
    if r < n and a[r] > a[largest]: largest = r
    # Wenn Root nicht das Groesste ist...
    if largest != i:
        # ...vertausche beide
        a[i], a[largest] = a[largest], a[i]
        # Max-Heap-Eigenschaft wiederherst.
        heapify(a, n, largest)
```

```
def heap_sort(a):
    n = len(a)
    # Baue einen Max-Heap auf (bottom-up)
    for i in range(n//2 - 1, -1, -1):
        heapify(a, n, i)
    # Elemente einzeln entnehmen
    for i in range(n-1, 0, -1):
        # Aktuelle Wurzel ans Ende setzen
        a[i], a[0] = a[0], a[i]
        # Erneut Max-Heap aufbauen
        heapify(a, i, 0)
```

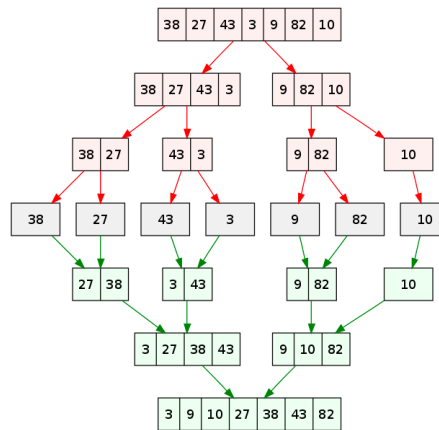


4.7. Binary Sort

Eine Variante von Insertion Sort:

- Findet die Einfügeposition per binärer Suche.
- Die Verschiebung der Elemente bleibt aber linear.
- **Laufzeit** — $\Theta(n^2)$ (Vergleiche: $\Theta(n \log n)$, Verschiebungen: $\Theta(n^2)$).

```
def binary_insertion_sort(a):
    # Starte beim zweiten Element
    for i in range(1, len(a)):
        # Element das eingefuegt wird
        key = a[i]
        # Grenzen fuer binaere Suche
        low, high = 0, i - 1
        # Suche richtige Position
        while low <= high:
            mid = (low + high) // 2
            # Suche in linker Haelfte
            if key < a[mid]: high = mid - 1
            # Suche in rechter Haelfte
            else: low = mid + 1
        # Verschiebe alle groesseren Elem...
        for j in range(i, low, -1):
            # ...eins nach rechts
            a[j] = a[j - 1]
        # Setze Element an gefundene Pos
        a[low] = key
```



5. Asymptotische Laufzeit

5.1. Master-Theorem

Das Master-Theorem löst Rekurrenzen der Form $T(n) = a \cdot T(n/t) + f(n)$.
Vergleiche dazu die „äußere Arbeit“ $f(n)$ mit der Laufzeit der Rekursion $n^{\log_t(a)}$:

Fall ($f(n)$ vs $n^{\log_t(a)}$)	Intuition	Ergebnis $T(n)$
$f(n)$ ist polynomiell kleiner	Rekursion dominiert	$\Theta(n^{\log_t(a)})$
beide sind gleich ($f(n) \in \Theta(n^{\log_t(a)})$)	Balance	$\Theta(n^{\log_t(a)} \cdot \log n)$
$f(n)$ ist polynomiell größer	Arbeit außen dominiert	$\Theta(f(n))$

Hinweis: „Polynomiell“ bedeutet, dass der Unterschied mindestens ein Faktor n^ϵ (mit $\epsilon > 0$) sein muss, ein bloßer $\log n$ -Faktor reicht nicht aus. Im 3. Fall (größer) muss formal zudem die Regularitätsbedingung erfüllt sein.

5.2. Häufige Summen

Summe	Vereinfachung	Laufzeit (Θ)
$\sum_{i=1}^n 1$	n	$\Theta(n)$
$\sum_{i=1}^n i$	$\frac{n(n+1)}{2}$	$\Theta(n^2)$
$\sum_{i=1}^n i^2$	$\frac{n(n+1)(2n+1)}{6}$	$\Theta(n^3)$
$\sum_{i=1}^n i^3$	$\left(\frac{n(n+1)}{2}\right)^2$	$\Theta(n^4)$
$\sum_{i=0}^n 2^i$	$2^{n+1} - 1$	$\Theta(2^n)$
$\sum_{i=0}^n a^i$ ($a > 1$)	$\frac{a^{n+1} - 1}{a - 1}$	$\Theta(a^n)$
$\sum_{i=1}^n \frac{1}{i}$	$\ln n + \gamma$ (harmonisch)	$\Theta(\log n)$
$\sum_{i=1}^{\log n} i$	$\frac{(\log n)(\log n + 1)}{2}$	$\Theta((\log n)^2)$
$\sum_{i=1}^n \log i$	$\log n! \approx n \log n$	$\Theta(n \log n)$
$\sum_{i=1}^n \sqrt{i}$	$\approx \frac{2}{3} n^{3/2}$	$\Theta(n^{3/2})$
$\sum_{i=1}^n i \log i$	keine einfache Formel	$\Theta(n \log n)$
$\sum_{i=1}^n 2^{n-i}$	$2^n \cdot \sum_{i=1}^n 2^{-i} \approx 2^n$	$\Theta(2^n)$

5.3. Laufzeiten klassischer Schleifenmuster

```
for i in range(n):
    f()
```

Aufrufe: n
Laufzeit: $\Theta(n)$

```
for i in range(n):
    for j in range(n):
        f()
```

Aufrufe: n^2 Laufzeit: $\Theta(n^2)$ **Konstante Grenze:** $\Theta(n)$

```
for i in range(n):
    for j in range(i):
        f()
```

Aufrufe: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ Laufzeit: $\Theta(n^2)$

```
for i in range(n):
    for j in range(i, n):
        f()
```

Aufrufe: $\sum_{i=0}^{n-1} (n-i)$ Laufzeit: $\Theta(n^2)$

```
for i in range(n):
    for j in range(n):
        for k in range(n):
            f()
```

Aufrufe: n^3 Laufzeit: $\Theta(n^3)$

```
for i in range(n):
    for j in range(i):
        for k in range(j):
            f()
```

Aufrufe: $\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} j$ Laufzeit: $\Theta(n^3)$

```
for i in range(n):
    for j in range(n):
        for k in range(j):
            f()
```

Aufrufe: $n \cdot \sum_{j=0}^{n-1} j$ Laufzeit: $\Theta(n^3)$

Wurzeln

$\Rightarrow$ Enthält die Abbruchbedingung eine Potenz der Laufvariable, umformen: $m \cdot m < n \Rightarrow m < \sqrt{n}$.

```
m = 0
while m * m < n:
    f()
    m += 1
```

Aufrufe: $\lceil \sqrt{n} \rceil$ Laufzeit: $\Theta(\sqrt{n})$

Substitution von Obergrenzen

$\Rightarrow$ Ist die äußere Grenze n^2 statt n : bekanntes Muster anwenden und $n \rightarrow n^2$ einsetzen.

```
# Muster: Grenze n -> Theta(n^2)
# Hier: Grenze n^2 -> Theta((n^2)^2) = Theta(n^4)
for i in range(n * n):
    for j in range(i):
        f()
```

Aufrufe: $\sum_{i=0}^{n^2-1} i = \frac{1}{2} n^2 (n^2 - 1)$ Laufzeit: $\Theta(n^4)$

5.4. Logarithmische und gemischte Muster

Schlüsselprinzip

$\Rightarrow$ Exponentielle Änderung von i oder $n \Rightarrow$ logarithmische Laufzeit.

```
i = n
while i > 1:
    f()
    i //= 2
```

Aufrufe: $\log n$ Laufzeit: $\Theta(\log n)$

Vorwärts-Logarithmus

$\Rightarrow$ Multiplikation mit konstantem Faktor > 1 ist äquivalent zur Division ($i \neq 2$) — ebenfalls logarithmisch.

```
i = 1
while i < n:
    f()
    i = i * 2
```

Aufrufe: $\lceil \log_2(n) \rceil$ Laufzeit: $\Theta(\log n)$

```
for i in range(n):
    j = i
    while j > 0:
        f()
        j //= 2
```

Aufrufe: $\sum_{i=1}^n \log i$ Laufzeit: $\Theta(n \log n)$

```
for i in range(n):
    for j in range(n):
        k = j
        while k > 0:
            f()
            k //= 2
```

Aufrufe: $n^2 \log n$ Laufzeit: $\Theta(n^2 \log n)$

5.5. Besondere Muster

```
for i in range(n):
    for j in range(i*i):
        f()
```

Aufrufe: $\sum_{i=0}^{n-1} i^2$ Laufzeit: $\Theta(n^3)$

```
for i in range(n):
    for j in range(1, i+1):
        if i % j == 0:
            f()
```

Aufrufe: $\sum_{i=1}^n \log i$ Laufzeit: $\Theta(n \log n)$

```
for i in range(n):
    for j in range(n):
        for k in range(i + j):
            f()
```

Aufrufe: $\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} (i+j)$ Laufzeit: $\Theta(n^3)$

5.6. Exponentielle Rekursion

```
def f(n):
    if n <= 1:
        return
    f(n-1)
    f(n-1)
```

Aufrufe: 2^n Laufzeit: $\Theta(2^n)$

```
def f(n):
    if n <= 1:
        return
    for i in range(n):
        f(n-1)
```

Aufrufe: $n \cdot f(n-1)$ Laufzeit: $\Theta(n!)$

```
def h(n):
    if n <= 1:
        return
    # rekursiver Aufruf mit Halbierung
    h(n // 2)
```

Aufrufe: $\lceil \log_2 n \rceil + 1$ Laufzeit: $\Theta(\log n)$

5.7. Asymptotische Laufzeit von Rekursionen

1. Gegebene Rekurrenz

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad T(1) = c$$

(Annahme: n ist eine Potenz von b , für sauberere Algebra.)

2. Rekurrenz auflösen (ausrollen)

$$\begin{aligned} T(n) &= aT\left(\frac{n}{b}\right) + f(n) \\ &= a \left[aT\left(\frac{n}{b^2}\right) + f\left(\frac{n}{b}\right) \right] + f(n) \\ &= a^2T\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n) \\ &= a^kT\left(\frac{n}{b^k}\right) + \sum_{i=0}^{k-1} a^i f\left(\frac{n}{b^i}\right). \end{aligned}$$

3. Abbruchbedingung

Stoppe, wenn $\frac{n}{b^k} = 1 \Rightarrow k = \log_b n$.

4. Einsetzen

$$T(n) = a^{\log_b n} T(1) + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right).$$

Beachte, dass $a^{\log_b n} = n^{\log_b a}$, womit die Summe wird zu

$$\sum_{i=0}^{\log_b n - 1} \Theta\left(n^d \left(\frac{a}{b^d}\right)^i\right),$$

einer geometrischen Reihe mit Quotient $r = \frac{a}{b^d}$.

6. Numpy

6.1. Array erzeugen

```
np.array([1, 2, 3]) # Array aus Python-Liste erstellen
np.asarray(x) # kein Copy falls x schon Array
np.copy(x) # tiefe Kopie: unabh. vom Original
np.zeros((3, 3)) # 3x3-Matrix, alle Werte = 0.0
np.ones((2, 4)) # 2x4-Matrix, alle Werte = 1.0
np.full((2, 2), fill_value=9) # 2x2-Matrix, alle Werte = 9
# gr1xgr2-Array, alle Werte = None (dtype=object)
vk = np.full((gr1, gr2), fill_value=None)
np.eye(4) # Einheitsmatrix: 1 auf Hauptdiag.
np.arange(1, 10, 2) # [1,3,5,7,9] von 1 bis <10, Schr. 2
np.linspace(0, 1, 100) # 100 gleichm. Werte von 0 bis 1
np.logspace(0, 3, 4) # [1,10,100,1000]: 10^0 bis 10^3
np.empty((2, 2)) # nicht init. --zuf. Speicherinhalt
```

6.2. Array-Informationen

```
a.shape # Tupel der Dimensionen (z.B. (3,4) = 3x4)
# Anzahl Dimensionen (2 fuer Matrix, 1 fuer Vektor)
a.ndim
a.size # Gesamtanzahl Elemente (Produkt der Dimensionen)
a.dtype # Datentyp der Elemente (z.B. int32, float64)
a.itemsize # Bytes pro Element (z.B. 4 fuer int32)
a.nbytes # Gesamtspeicher in Bytes (= size * itemsize)
a.T # Transponierte Matrix
a.strides # Schrittgroesse pro Achse in Bytes
a.flags # Speicherlayout-Flags (C_CONTIGUOUS, etc.)
a.base # Basisdaten: None=Owner, sonst Quelle des Views
```

6.3. Typumwandlung

```
a.astype(np.float32) # in float32 umwandeln (kopiert)
# kein Copy falls dtype schon passt
a.astype("int32", copy=False)
a.tolist() # zurueck zur Python-Liste
```

6.4. Indexing, Slicing, Maskierung

```
a[1], a[1, :], a[:, 0] # Element / ganze Zeile / ganze Spalte
a[1:4], a[:, 2] # Slicing / jedes 2. Element
a[a > 3] # boolescher Indexzugriff (Maske)
# bedingte Ersetzung: a wenn >0, sonst 0
np.where(a > 0, a, 0)
np.nonzero(a) # Indizes aller Nicht-Null-Elemente
np.take(a, [0, 2]) # Elemente per Index sammeln
# Elemente per Index setzen (in-place)
np.put(a, [1, 3], [9, 10])
np.compress(a > 2, a) # Elemente filtern per boolescher Maske
```

6.5. Broadcasting und Shapes

```
a + b # Shape automatisch erweitern
# resultierenden Shape bestimmen
np.broadcast_shapes(a.shape, b.shape)
np.broadcast_to(a, (3, 4)) # manuelles Broadcasting (View)
np.expand_dims(a, axis=0) # neue Achse einfüegen
np.squeeze(a) # Achsen der Groesse 1 entfernen
np.reshape(a, (2, -1)) # Form aendern (-1 = auto)
np.tile(a, (2, 3)) # Muster 2x vertikal, 3x horiz.
```

6.6. Stacking und Splitting

```
# Kombinieren von Arrays
np.concatenate((a, b), axis=0) # entlang bestehender Achse
np.stack([a, b], axis=0) # neue Achse einfüegen
np.vstack([a, b]) # vertikal stapeln (Zeilen)
np.hstack([a, b]) # horizontal stapeln (Spalten)
np.dstack([a, b]) # tiefen-stapeln (3. Achse)
# Aufteilen von Arrays
np.split(a, 3) # in 3 gleiche Teile aufteilen
np.array_split(a, 3) # in 3 Teile (ungleiche OK)
np.vsplit(a, 2) # vertikal in 2 Haelften teilen
np.hsplit(a, 2) # horizontal in 2 Haelften teilen
```

6.7. Mathematik

```
np.add(a, b), np.subtract(a, b) # Addition, Subtraktion
np.multiply(a, b), np.divide(a, b) # Multiplikation, Division
np.mod(a, b), np.remainder(a, b) # Modulo / Rest
np.exp(a), np.log(a), np.log10(a) # e^a, ln(a), log10(a)
np.sqrt(a), np.square(a) # Wurzel, Quadrat (a^2)
np.clip(a, min, max) # Werte auf [min, max] begrenzen
np.sign(a), np.abs(a) # Vorzeichen (-1/0/1), Betrag
# Aufrunden, Abrunden, Runden
np.ceil(a), np.floor(a), np.round(a)
```

6.8. Aggregate und Statistik

```
np.sum(a), np.prod(a) # Summe, Produkt aller Elemente
np.mean(a), np.median(a) # arithm. Mittelwert, Median
np.std(a), np.var(a) # Standardabweichung, Varianz
np.min(a), np.max(a) # kleinster, groesster Wert
np.argmin(a), np.argmax(a) # Index von Min bzw. Max
np.ptp(a) # Peak-to-Peak: max(a) - min(a)
np.percentile(a, 75) # 75. Perzentil
np.histogram(a, bins=10) # Histogramm berechnen
```

7. Pandas

7.1. DataFrame erzeugen und darauf zugreifen

```
import pandas as pd
df = pd.DataFrame() #leerer DataFrame
df = pd.read_csv("Name.csv") #DataFrame aus CSV laden
#Zugriff auf Daten
data["Month"] #einzelne Spalte
data[["Name", "City"]] #mehrere Spalten
data["Name"][2] #gibt 'Carlos' zurück
data.iloc[1] #Zeile mit Index 1
data[1:3] #Zeilen mit Index 1 und 2
data.loc[1:3, "M":"C"] #Zeilen 1--3, Spalten M bis C
data.iloc[1:3, 0:2] #wie loc, aber indexbasiert
```

7.2. Spalten umbenennen

```
data.set_index("Row") #setzt 'Row' als Index
data.rename(columns={City: Location}) #Spalte umbenennen
#jede Spalte umbenennen
data.columns = ["Monat", "Name", "Stadt", ...]
```

7.3. Filtern von Zeilen

```
data[data["Age"] > 30] #alle Personen über 30
data["Name"][data["Age"] > 30] #nur Namen dieser Personen
```

7.4. Zeilen und Spalten löschen

```
data.drop(columns=["Age"]) #löscht Spalte 'Age'
data.drop(data.index[0]) #löscht erste Zeile
#löscht Zeilen mit NaN
data.dropna(axis=0, how="any") #wenn mind. 1 NaN in der Zeile
data.dropna(axis=0, how="all") #wenn alle NaN in der Zeile
#löscht Spalten mit NaN
data.dropna(axis=1, how="any") #wenn mind. 1 NaN in der Spalte
```

7.5. Hilfsfunktionen für DataFrames

```
data["Age"].sum() #summiert alle Werte der Spalte 'Age'
data["Age"].max() #maximaler Wert
data.fillna(...) #ersetzt NaN durch gewünschten Wert
```

8. ML

1. Daten einlesen, Modell trainieren und validieren

Dieser Code zeigt die Grundschriffe eines typischen ML-Prozesses: Einlesen der Daten, Aufteilen in Trainings- und Testdaten, Trainieren eines Klassifikationsmodells (z.B. Decision Tree), sowie Validieren mittels Accuracy oder R2-Score.

```
import pandas as pd
import numpy as np # optional

# Daten einlesen und in X und y aufteilen
df = pd.read_csv("data.csv")
X = df.drop(["target"], axis=1)
y = df["target"]

# in Trainings- und Testdaten aufteilen
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Modell trainieren
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# Klassifikationsmodell validieren
from sklearn.metrics import accuracy_score
y_pred = model.predict(X_test)
validation_score = accuracy_score(y_test, y_pred)

# oder Regressionsmodell validieren
from sklearn.metrics import r2_score
validation_score = r2_score(y_test, y_pred)
```

2. Vorhersage mit neuen Daten

Sobald ein Modell trainiert ist, kann es mit neuen Eingabedaten (X_final) verwendet werden, um Vorhersagen (y_final) zu erzeugen.

```
X_final = pd.read_csv("X_final.csv")
y_final = model1.predict(X_final)
return y_final
```

3. Neuronales Netz mit MLPClassifier

Ein Beispiel für ein neuronales Netz (Multilayer Perceptron). Die Architektur wird über hidden_layer_sizes definiert, z.B. fünf versteckte Schichten mit je 6–8 Neuronen. (→ 10.1)

```
from sklearn.neural_network import MLPClassifier
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=31)
```

```
model = MLPClassifier(
    hidden_layer_sizes=(8, 6, 7, 6, 7, 5),
    max_iter=5000,
    random_state=85)
```

```
model.fit(X_train, y_train)
```

8.1. Wichtige Parameter

- **max_iter** — Maximale Anzahl Iterationen, die ein Modell während des Trainings durchläuft, um die Verlustfunktion zu minimieren (z. B. Training endet, wenn Loss < 0.001).
Verwendet von: LogisticRegression, MLPClassifier, MLPRegressor
- **hidden_layer_sizes** — Gibt die Struktur eines neuronalen Netzes an (z. B. Anzahl Neuronen pro Schicht).
Verwendet von: MLPClassifier, MLPRegressor
- **random_state** — Initialisiert den Zufallsprozess (z. B. Shuffle bei train_test_split, Gewichtung im Netz), um reproduzierbare Ergebnisse zu garantieren.
Verwendet von: allen Modulen mit Zufallskomponenten
- **test_size** — Prozentualer Anteil der Daten, der für die Testmenge reserviert wird. Üblich ist 0.1 bis 0.3.
Verwendet von: train_test_split
- **cv (Cross-Validation)** — Gibt bei GridSearchCV oder cross_val_score die Anzahl an Folds an, mit denen K-Fold-Cross-Validation durchgeführt wird.
Verwendet von: GridSearchCV, cross_val_score, etc.
- **n_clusters** — Gibt an, wie viele Cluster ein Clustering-Algorithmus bilden soll. Wichtig bei unüberwachtem Lernen.
Verwendet von: KMeans, DBSCAN (implizit), AgglomerativeClustering

Grid Search via Cross-Validation

- **Normales Grid Search** — Du probierst verschiedene Modelltiefen und weitere Hyperparameter-Kombinationen aus. Für jede Kombination misst du die Fehlerrate des Modells.
- **k-Fold-Cross-Validation** — Die Daten werden in k gleich grosse Teile (Folds) aufgeteilt. Das Modell wird k -mal trainiert — jedes Mal mit $k - 1$ Folds als Trainingsdaten und dem verbleibenden Fold als Validierungsmenge. Jeder Fold dient genau einmal als Validierung, sodass eine zuverlässige Schätzung der Modellgüte entsteht.

8.2. Modelle

ML-Typen: Regression vs. Klassifikation

- **Regression** — wird verwendet, um stetige Werte vorherzusagen (z. B. Preis, Wahrscheinlichkeit) in '%’.
- **Klassifikation** — hingegen ordnet Datenpunkte einer festen Klasse zu (z. B. Spam/Nicht-Spam), ja oder nein.

Modelle für Regression

Linear Regression

- Vorhersage basiert auf einer linearen Kombination der Eingabefeatures:

y = w1x1 + w2x2 + ... + wn xn + b

- **Import** — from sklearn.linear_model import LinearRegression

Decision Tree Regressor

- Modelliert nicht-lineare Beziehungen durch Entscheidungen an Verzweigungsknoten.
- Zerlegt die Eingabemenge in Zonen mit möglichst homogener Zielvariable.
- **Import** — from sklearn.tree import DecisionTreeRegressor

Neural Network Regressor

- Ein neuronales Netz mit Regressionsziel: nicht-linear, viele Parameter.
- Flexible Anpassung durch hidden_layer_sizes, max_iter, activation.
- **Import** — from sklearn.neural_network import MLPRegressor

Bewertung mit dem R2-Score

Der R²-Score misst die Qualität einer Regressionsvorhersage:

R^2(D, f) = 1 - MSE(D, f) / MSE(D, f_0)

- **MSE hängt von der Skalierung der Daten ab, R² - Score nicht**
- f₀ — konstante Durchschnittsvorhersage
- D — Datensatz
- MSE — Mean Squared Error, misst mittlere quadratische Abweichung (Verlustfunktion):

MSE(D, f) = 1/n * sum_{i=1}^n (y_i - y_hat_i)^2, y_hat_i = f(x_i)

LinearRegression

- **Wann einsetzen** — Für einfache Regressionsprobleme, wenn der Zusammenhang zwischen Features und Zielwert annähernd linear ist.
- **Stärken** — schnell, gut interpretierbar, theoretisch gut verstanden.
- **Schwächen** — empfindlich gegenüber Ausreißern, setzt Linearität voraus, kann komplexe Muster nicht abbilden.
- **Wichtige Hyperparameter** — fit_intercept: ob ein Bias-Term berücksichtigt wird. n_jobs: Anzahl CPU-Kerne, die während des Fits genutzt werden.
- **Beispiel-Use-Case** — Vorhersage von Hauspreisen anhand von Features wie Wohnfläche, Anzahl Zimmer und Baujahr in einer kleinen Nachbarschaft.

from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

Modelle für Klassifikation

Voraussetzung

Die Daten müssen gelabelt sein.

Logistic Regression

- Verwendet die **sigmoid**-Funktion zur Klassenvorhersage:

y = sigma(w_0 + w_1x_1 + ... + w_nx_n)

- Ausgabe ist die Wahrscheinlichkeit, dass ein Punkt zur positiven Klasse gehört.
- **Import** — from sklearn.linear_model import LogisticRegression

Decision Tree Classifier

- Genau wie der Regressor, aber mit Klassenzuordnung am Ende der Blätter.
- Tiefe entspricht max. Anzahl der Entscheidungen
- Gut interpretierbar, aber anfällig für **Overfitting**.
- **Import** — from sklearn.tree import DecisionTreeClassifier

Neural Network Classifier

- Tieferes neuronales Netz, spezialisiert auf Klassifikationsaufgaben.
- Meist verwendet: MLPClassifier (Multilayer Perceptron)
- **Import** — from sklearn.neural_network import MLPClassifier

(Balanced) Accuracy

Confusion Matrix

real / vorhergesagt	Positiv	Negativ
Positiv	True Pos. (TP)	False Neg. (FN)
Negativ	False Pos. (FP)	True Neg. (TN)

Accuracy — Anteil der korrekt klassifizierten Beobachtungen:

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Balanced Accuracy — Mittelwert der True Positive Rate (TPR) und der True Negative Rate (TNR): Aka Zeilen werden Normiert (Summe jeder Zeile ist 1)

Balanced Accuracy = 1/2 * (TPR + TNR)

mit:

TPR = TP / (TP + FN) und TNR = TN / (TN + FP)

Hinweis

Die Balanced Accuracy ist nützlich bei unausgeglichenen Klassenverteilungen, da beide Klassen gleich stark berücksichtigt werden.

0-1-Loss:

Die 0-1-Loss-Funktion misst, ob eine Vorhersage korrekt ist:

L(y, y_hat) = { 0, wenn y = y_hat; 1, wenn y != y_hat }

Für einen Datensatz mit n Beispielen:

Empirical Risk = 1/n * sum_{i=1}^n L(y_i, y_hat_i)

Im Bspl der Confusion Matrix:

0-1-Loss (Error Rate) = (FP + FN) / (TP + TN + FP + FN)

Die Accuracy ist das Komlement zum Loss

Accuracy = 1 - Loss = (TP + TN) / (TP + TN + FP + FN)

Gini-index Loss:

Gini-Index (als Loss im Entscheidungsbaum):

Die „Impurity“ einer Partition $Part_m$ ist:

$$G(Part_m) = \sum_k \hat{p}_{m,k}(1 - \hat{p}_{m,k}), \tag{8}$$

wobei $\hat{p}_{m,k}$ der Anteil der Klasse/Labels k in der Partition ist.
Ein **Split** teilt den Knoten $Part_m$ in z.B. zwei Partitionen $Part_1$ und $Part_2$.
Der Gini-Index für den gesamten Split ist:

$$G_{\text{split}} = \frac{|Part_1|}{|Part_m|} G(Part_1) + \frac{|Part_2|}{|Part_m|} G(Part_2). \tag{9}$$

wobei $|Part_i|$ die Anzahl der Datenpunkte in der Partition i ist

Hinweis
Ein Entscheidungsbaum wählt den Split, der den Gini-Index minimiert.

LogisticRegression

- **Wann einsetzen** — Für binäre oder multinomiale Klassifikation, wenn die Entscheidungsgrenze annähernd linear ist. Liefert Wahrscheinlichkeitsschätzungen.
- **Stärken** — interpretierbar, schnell, liefert Klassenwahrscheinlichkeiten.
- **Schwächen** — schwierig bei nicht-linearen Mustern.
- **Wichtige Hyperparameter** — max_iter: ca. 500–1000, class_weight: "balanced", None.
- **Beispiel-Use-Case** — Vorhersage, ob ein Patient Diabetes hat (ja/nein), basierend auf medizinischen Messwerten — wenn die Features einen linearen Zusammenhang aufweisen.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter = 1000,
                           "class_weight" = balanced)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

DecisionTreeClassifier

- Faustregeln**
- **max_depth** — 3–10 für kleine/mittlere Datensätze. Beschränken, um Overfitting zu vermeiden.
 - **min_samples_split** — erhöhen (z.B. 10+), um zu regularisieren.
 - **min_samples_leaf** — 1–5, um Vorhersagen zu glätten.

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    # aus Vorlesung
    # für Reproduzierbarkeit
    random_state=0
    # Modellkomplexität steuern
    max_depth=4,
    # optional
    # "entropy" als Alternative
    criterion="gini",
    splitter="best",      # oder "random"
    min_samples_split=2,  # Standard
    min_samples_leaf=1,   # Standard
    # "sqrt" oder "log2" probieren
    max_features=None,
)
model.fit(X_train, y_train)
```

MLPClassifier

- Faustregeln**
- **hidden_layer_sizes** — 1–2 Schichten mit je 50–100 Neuronen funktionieren oft gut.
 - **activation** — "relu" verwenden. "tanh" bei kleinen/strukturierten Daten.
 - **alpha** — Default ist 0.0001. Erhöhen, um Overfitting zu reduzieren.
 - **max_iter** — hoch setzen (z.B. 500), falls das Training nicht konvergiert.

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(
    # zwei Schichten
    hidden_layer_sizes=(50, 50),
    # erhöhen falls nötig
    max_iter=5000,
    random_state=0
    # optional
    # oder "tanh", "logistic"
    activation="relu",
    # oder "sgd", "lbfgs"
    solver="adam",
    # L2-Regularisierung
    alpha=0.0001,
    # oder "adaptive"
    learning_rate="constant",
)
model.fit(X_train, y_train)
```

GridSearchCV

- Faustregeln**
- **param_grid** — 2–4 Werte pro Parameter.
 - **cv** — 5-Fold ist Standard. 3 für Geschwindigkeit, 10 für höhere Genauigkeit.
 - **scoring** — Metrik passend zur Aufgabe wählen.

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(random_state=0)

param_grid = {
    "max_depth": [2, 4, 6],
    "class_weight": [None, "balanced"]
    "min_samples_split": [2, 10],
    "min_samples_leaf": [1, 5]
}

search = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    # oder "accuracy", "recall", etc.
    scoring="balanced_accuracy",
    cv=5,
    verbose=1 # nur Terminalausgabe
)
search.fit(X_train, y_train)
```

8.3. Nicht-numerische Daten: Kategorische Merkmale encodieren

Maschinelles Lernen erfordert in der Regel numerische Eingabedaten. Nicht-numerische (kategorische) Daten wie z. B. \Farbe" oder \Beruf" müssen zuerst in eine numerische Form gebracht werden. Dazu gibt es verschiedene Kodierungsverfahren – je nach Kontext und Modell mit eigenen Vor- und Nachteilen:

Ordinal Encoding

- Pro:**
- Einfach umsetzbar.
 - Ermöglicht die Abbildung von geordneter Struktur (z. B. 1 = Crew, 2 = First Class).
- Con:**
- Kann eine falsche Nähe zwischen Kategorien suggerieren (z. B. 2 ist näher an 1 als 3, obwohl inhaltlich 1 und 3 ähnlicher sind).
 - Führt möglicherweise zu verzerrten Modellinterpretationen.

Mean Encoding

Pro:

- Nutzt den Mittelwert der Zielvariable pro Kategorie – kann nützliche Informationen übertragen.
- Versucht „scheinbare Nähe“ zwischen Kategorien auszugleichen.

Con:

- Gefahr von **Data Leakage**: Die Zielvariable „sickert durch“ in die Features.
- Erhöhtes Risiko von **Overfitting**.
- Verlangt Rechenaufwand.

One-hot Encoding

Pro:

- Höchste Präzision: Kodiert jede Kategorie in einer separaten Spalte.
- Besonders geeignet für nicht-ordinale Kategorien.

Con:

- Jeder neue Wert erzeugt eine neue Spalte (hoher Speicherplatzverbrauch).
- Bei vielen Kategorien kann das Datenmodell sehr breit werden („Fluch der Dimensionalität“).

```
import pandas as pd
```

```
pd.get_dummies(a)
```

8.4. Dimensionsreduktion

Faustregeln

- **n_components** — 2–3 für Visualisierung, höher für Preprocessing.
- **PCA** — linear, schnell, maximiert Varianz. (→ 11.2)
- **t-SNE** — nichtlinear, gut für Cluster-Visualisierung, langsamer.

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
```

```
# --- Vorverarbeitung ---
# Normalisieren (bei Bilddaten üblich)
images_normalized = images / 255.0

# Oder Standardisieren (bei tabellarischen Daten üblich)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Bilder "flatten" (z.B. 28x28 → 784 Dimensionen)
images_flat = images.reshape(len(images), -1)
```

```
# --- Dimensionsreduktion ---
# PCA (linear)
model = PCA(n_components=2)

# oder t-SNE (nichtlinear, gleiche API)
# model = TSNE(n_components=2, random_state=0)
```

```
# Fit und Transformation
images_trans = model.fit_transform(images_flat)
```

8.5. Clustering

Faustregeln

- **StandardScaler** — immer vor PCA anwenden, wenn Features unterschiedliche Skalen haben.
- **n_components** — 2–3 für Visualisierung, mehr für Analyse.
- **K-Means** — `init="k-means++"` liefert stabilere Ergebnisse. (→ 11.1)

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# --- Daten laden ---
data, labels = load_digits(return_X_y=True)
(n_samples, n_features), n_digits = (
    data.shape, np.unique(labels).size)

# --- Vorverarbeitung ---
scaled = StandardScaler().fit_transform(data)

# --- PCA-Reduktion auf 2 Dimensionen ---
reduced_data = PCA(n_components=2).fit_transform(scaled)

# --- K-Means Clustering ---
kmeans = KMeans(init="k-means++", n_clusters=n_digits,
                n_init=4)
kmeans.fit(reduced_data)
```

8.6. Was tun, wenn der accuracy_score zu niedrig ist?

Wenn dein Klassifikationsmodell eine zu geringe Genauigkeit erreicht, kannst du systematisch folgende Maßnahmen ausprobieren:

1. Mehr Daten

- Größere Trainingsmengen helfen oft, Muster robuster zu erkennen.
- Dataaugmentation oder neue Datenquellen prüfen.

2. Feature Engineering

- Relevante Merkmale hinzufügen, irrelevante entfernen.
- Skalierung oder Transformation (z. B. StandardScaler).
- Kategorische Variablen korrekt encodieren (One-Hot, Ordinal, Mean).

3. Modellparameter optimieren

- `hidden_layer_sizes` beim MLPClassifier vergrößern oder anpassen.
- `max_iter` erhöhen (z. B. 2000 statt 500), damit das Modell besser konvergiert.
- `alpha` bei Regularisierung (MLP, Ridge...) anpassen.
- Andere activation Funktionen testen: `tanh`, `relu`...

4. Randomisierung kontrollieren

- `random_state` anpassen, wenn das Ergebnis stark schwankt.
- Mehrere Splits testen und Mittelwert bilden (z. B. mit `cross_val_score`).

5. Anderes Modell testen

- Statt Decision Tree → RandomForestClassifier oder MLPClassifier.
- Für lineare Probleme eventuell LogisticRegression.

6. Overfitting prüfen

- Wenn Training sehr gut, aber Test schlecht: Modell zu komplex?
- `max_depth` bei Trees begrenzen, dropout oder `alpha` bei MLP verwenden.

9. Datenstrukturen

9.1. Queue und Stack

Queue

Eine Warteschlange, bei der Elemente nach dem *First In – First Out* (FIFO) Prinzip verarbeitet werden.

- **add/enqueue** — Element am Ende der Queue einfügen
- **remove/dequeue** — Element am Anfang der Queue entfernen

Stack

Ein Stapelspeicher, der nach dem *Last In – First Out* (LIFO) Prinzip arbeitet.

- **add/push** — Element oben auf den Stack legen
- **remove/pop** — Oberstes Element entfernen

```
stack = []                # Stack (LIFO)
stack.append(x)           # push   → O(1)
top = stack.pop()         # pop    → O(1)
```

```
from collections import deque # Queue (FIFO)
q = deque()
q.append(x)                   # enqueue → O(1)
first = q.popleft()           # dequeue → O(1)
# list.pop(0) wäre O(n) → deque benutzen!
```

9.2. Binary (Search) Trees

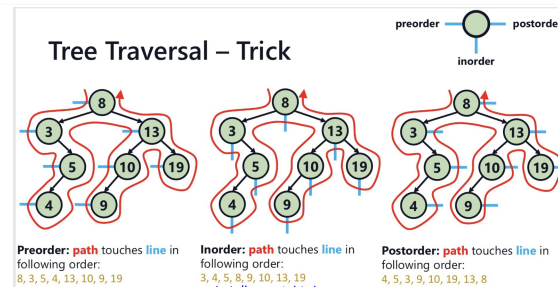
Baumdurchläufe (Traversal)

```
def preorder_list(node: Optional[Node]) → list:
    if node is None:
        return []
    return ([node.key] + preorder_list(node.left)
            + preorder_list(node.right))

def postorder_list(node: Optional[Node]) → list:
    if node is None:
        return []
    return (postorder_list(node.left)
            + postorder_list(node.right) + [node.key])

def inorder_list(node: Optional[Node]) → list:
    if node is None:
        return []
    return (inorder_list(node.left)
            + [node.key] + inorder_list(node.right))
```

Tree Traversal – Trick



- **Preorder** — Hauptreihenfolge, mit dem wird Baum gebaut
- **Postorder** — Nebenreihenfolge

Knoten suchen (search)

Durchschnittsfall: $\Theta(\log n)$, Worst Case: $\Theta(n)$

```
def search(self, key: int) → Optional[Node]:
    """Search for the node with the specified key
    in the tree and return it, or None if there
    is no such node."""
    n = self.root # Starte bei der Wurzel
    # Laufe, bis Schlüssel gefunden oder Ende erreicht
    while n is not None and n.key is not key:
        # Gesuchter Schlüssel ist kleiner → links weiter
        if key < n.key:
            n = n.left # Gehe zum linken Kind
        else:
            n = n.right # Sonst nach rechts gehen
    return n # Gefundener Knoten oder None
```

Knoten hinzufügen (add)

Durchschnittsfall: $\Theta(\log n)$, Worst Case: $\Theta(n)$

```
def add(self, key: int) → bool:
    """Add a node with the specified key to the
    tree. Return True if the node was added,
    and False otherwise."""
    if self.root is None:
        self.root = Node(key)
        return True
    n = self.root
    while n.key is not key:
        if key < n.key:
            if n.left is None:
                n.left = Node(key)
                return True
            n = n.left
        else:
            if n.right is None:
                n.right = Node(key)
                return True
            n = n.right
    return False
```

Knoten löschen (delete)

In allen Fällen: $\Theta(n)$

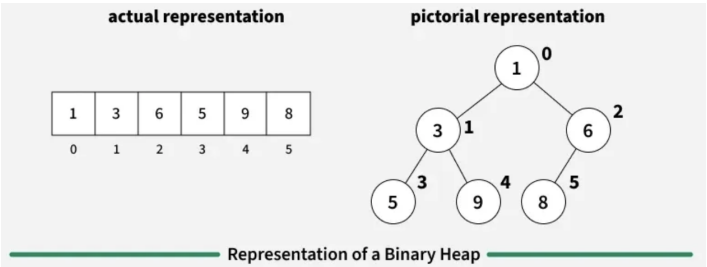
```
def deleteNode(root, key):
    if root is None:
        return None
    if key < root.val:
        root.left = deleteNode(root.left, key)
    elif key > root.val:
        root.right = deleteNode(root.right, key)
    else:
        # Case 1: No children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Only right child
        elif root.left is None:
            root = root.right
        # Case 2: Only left child
        elif root.right is None:
            root = root.left
        # Case 3: Two children
        else:
            successor = findSuccessor(root)
            root.val = successor.val
    return root

def findSuccessor(start_node):
    parent = start_node
    node = start_node.right
    while node.left is not None:
        parent = node
        node = node.left
    parent.left = node.right
    return node
```

Komplexitäten

- **Baum aufbauen (Worst Case)**— $\Theta(n^2)$ (balanciert: $\Theta(n \log n)$)
- **Minimum/Maximum finden**— $\mathcal{O}(n)$ (balanciert: $\mathcal{O}(\log n)$)
- **Einfügen**— $\mathcal{O}(n)$ (balanciert: $\mathcal{O}(\log n)$)
- **Entfernen (beliebiges Element)**— $\mathcal{O}(n)$ (balanciert: $\mathcal{O}(\log n)$)

9.3. Heaps



Grundidee eines Binary Heaps

Binary Heap

Ein Binary Heap ist eine vollständige Binärstruktur, bei der gilt:

- **Min-Heap** — Elternknoten ist stets kleiner als seine Kinder.
- **Max-Heap** — Elternknoten ist stets größer als seine Kinder.

Heaps werden in einer Liste gespeichert, sodass die Knotenebenen hintereinander notiert sind:

[1, 2, 3, 4, 5, 6] $\Rightarrow$ Index: 0,1,2,...

Zugriffsformeln

- **parent** — $(i - 1) // 2$
- **left child** — $2i + 1$
- **right child** — $2i + 2$
- **Höhe** — $\log_2(N + 1)$ mit $N = \text{len}(\text{list})$

Heapify (Heap aufbauen)

```
def heapify(list):
    for i in range(len(list)):
        while list[(i - 1)//2] > list[i] and i > 0:
            swap(list, (i - 1)//2, i)
            i = (i - 1)//2
    return
```

oder:

```
def heapify(list):
    n = len(list)
    for i in range(n//2 - 1, -1, -1):
        shift_down(list, i, n)
    return
```

SiftDown (oberstes Element löschen)

Beim Entfernen der Wurzel (z.B. Max-Element bei Max-Heap) wird das letzte Element nach oben geschoben und mit seinen Kindern verglichen. Dabei wird immer mit dem größeren (bzw. kleineren bei Min-Heap) Kind getauscht.

```
def SiftDown(a, i, m):
    while 2*i + 1 < m:
        j = 2*i + 1
        if j + 1 < m and a[j] < a[j + 1]:
            j += 1
        if a[i] >= a[j]:
            break
        a[i], a[j] = a[j], a[i]
        i = j
```

Sortieren mit Heaps (Heap Sort)

```
def SortHeap(a):
    heapify(a)
    for i in reversed(range(len(a))):
        swap(a[0], a[i])
        SiftDown(a, 0, i)
    return
```

Elemente einfügen und löschen

- **Einfügen** — ans Ende der Liste hängen, dann siftUp()
- **Löschen** — Element mit letztem tauschen, löschen, dann SiftDown()

```
def removeMax(heap):
    if len(heap) == 0:
        return None
    max_val = heap[0]
    heap[0] = heap[-1]
    heap.pop()
    SiftDown(heap, 0, len(heap))
    return max_val
```

Komplexitäten

- **Heap aufbauen (Worst Case)** — $\Theta(n \log n)$
- **Maximum/Minimum holen** — $\mathcal{O}(1)$
- **Einfügen in Heap** — $\mathcal{O}(\log n)$
- **Entfernen (Top-Element)** — $\mathcal{O}(\log n)$

9.4. Hash-Tabellen

Grundidee

Hash-Tabelle

Eine Hash-Tabelle speichert ungeordnete Sammlungen **einzigartiger Elemente** (z. B. set oder dict in Python).

Hashing: Einer Eingabe wird mittels **Hash-Funktion** ein Integer zugewiesen. Die Platzierung im Array der Länge M erfolgt am Index:

$$\text{index} = \text{hash}(x) \% M \quad (\text{math: mod})$$

Laufzeiten

- **Suche, Einfügen, Entfernen (Erwartungswert)** — $\mathcal{O}(1)$
- **Suche, Einfügen, Entfernen (Worst Case)** — $\mathcal{O}(n)$

Kollisionsbehandlung

Treffen mehrere Elemente auf denselben Index, gibt es zwei Lösungsansätze:

- **Chaining** — Am Index wird der Head-Knoten einer **Linked List** gespeichert, die alle kollidierenden Elemente aufnimmt. (→ 12.4)
Vorteil: Ein oft wiederholter Hash blockiert nicht die restliche Tabelle.
- **Probing** — Ist der Index besetzt, wird zirkulär der nächste freie Index (**Wrap-around**) gesucht.
Vorteil: Alle Daten bleiben Cache-freundlich im Array.
Nachteil: Löschungen erfordern **Tombstoning** (spezielle Literale als Platzhalter) oder komplexes Rückwärts-Verschieben.

```
def our_hash(word):
    return ord(word[0]) # einfache Hash-Fkt.

def create_hash_table(l, M):
    res = [[] for _ in range(M)] # M leere Buckets
    for x in l:
        idx = our_hash(x) % M # % statt mod!
        if x not in res[idx]: # keine Duplikate
            res[idx].append(x) # chaining: anhaengen
    return res
```

9.5. Quadrees

Grundidee

Quadtree

Quadrees sind **volle Quarternärbäume** (jeder Knoten hat entweder 0 oder exakt 4 Kinder) zur effizienten Verarbeitung **räumlicher Daten in 2D** (z. B. Rechteck-Abfragen).

Struktur eines Knotens

Ein Knoten speichert:

- untere linke Ecke l und obere rechte Ecke u
- **Maximalkapazität** m
- Liste p von Punkten
- 4 Referenzen auf Kinder-Quadrees c_0, c_1, c_2, c_3

```
class QuadTree:
    def __init__(self, l, u, m):
        self.l, self.u = l, u # untere/obere Ecke
        self.m = m # Kapazitaet
        self.p = [] # Punkte im Knoten
        self.c = [None]*4 # Kinder c0..c3
```

Einfügen & Split

- Ein Punkt wird zu p hinzugefügt.
- Überschreitet die Anzahl der Punkte die Kapazität m , wird der Knoten in vier Viertel partitioniert.
- Die Punkte aus p werden auf die Kinder aufgeteilt und das eigene p geleert.

Rechtecks-Abfrage (Range Query)

- Die Suche wächst logarithmisch mit der Anzahl der Punkte: $\mathcal{O}(\log n)$.
- Schneidet das Such-Rechteck einen Quadranten nicht, wird dieser Zweig **komplett ignoriert (Pruning)**.

```
def query(node, q_l, q_u):
    if not overlaps(node, q_l, q_u): # Pruning!
        return [] # Zweig ignorieren
    hits = [pt for pt in node.p # lokale Punkte
            if inside(pt, q_l, q_u)]
    for child in node.c: # rekursiv in Kinder
        if child is not None:
            hits += query(child, q_l, q_u)
    return hits
```

Bemerkung

Die **Konstruktion** des Baumes ist relativ aufwendig; der Einsatz lohnt sich daher erst bei **höherfrequentierten Abfragen**.

9.6. Übersicht: Laufzeiten im Vergleich

Datenstruktur	Suche	Einfügen	Entfernen
Liste / Array	$\mathcal{O}(n)$	$\mathcal{O}(n)$ (worst)	$\mathcal{O}(n)$ (worst)
Linked List	$\mathcal{O}(n)$	$\mathcal{O}(1)$ (nach Suche)	$\mathcal{O}(1)$ (nach Suche)
BST (balanciert)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
BST (unbal.)	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Heap	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$ (Top)
Hash-Tabelle	$\mathcal{O}(1)$ (erw.)	$\mathcal{O}(1)$ (erw.)	$\mathcal{O}(1)$ (erw.)
Quadtree	$\mathcal{O}(\log n)$ (Range)	$\mathcal{O}(\log n)$	—

Merke

- Hash-Tabelle** — schnellster Zugriff im Durchschnitt, aber ungeordnet.
- BST / Heap** — geordneter Zugriff, log. Laufzeit — nur balanciert effizient.
- Quadtree** — spezialisiert auf 2D-Bereichs-Abfragen.

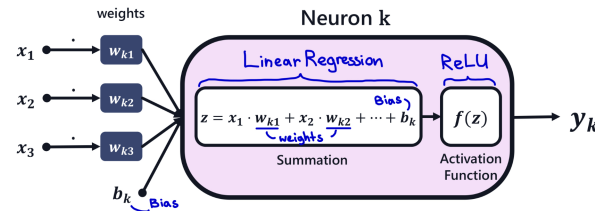
10. Neuronale Netzwerke

Neuronale Netzwerke (Neural Networks, NNs) bestehen aus vielen kleinen Funktionseinheiten, den sogenannten **Neuronen**. Diese eignen sich besonders gut zur Erkennung komplexer Muster in strukturierten Daten, Bildern oder Text.

A Neuron is:

Mathematical function which takes inputs and computes outputs

$$y_k = f(x_1 \cdot w_{k1} + x_1 \cdot w_{k2} + \dots + b_k)$$



10.1. Standard Neural Network (MLP)

Ein Neuron berechnet:

$$y_k = f(x_1 \cdot w_{k1} + x_2 \cdot w_{k2} + \dots + x_n \cdot w_{kn} + b_k)$$

- x_i — Eingabewerte
- w_{ki} — Gewichtung für Neuron k
- b_k — Bias-Term
- f — Aktivierungsfunktion, z. B.:
 - ReLU:** $f(x) = \max(0, x)$ (oft in versteckten Schichten)
 - Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$ (Ausgabe bei binärer Klassifikation)
- Gewichtungen w_{ki} und Bias b_k werden im Training angepasst

10.2. Softmax-Schicht (Ausgabe)

- Wandelt Ausgaben in Wahrscheinlichkeiten um:

$$P_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad \text{mit} \quad \sum_i P_i = 1$$

- Wird bei Mehrklassenklassifikation als Ausgangsschicht genutzt

10.3. Convolutional Neural Network (CNN)

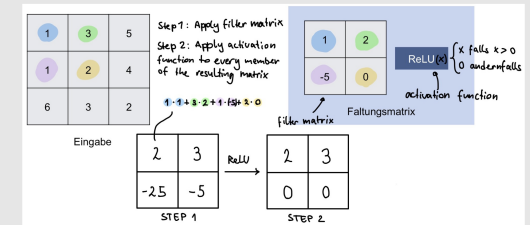
Ein Neuron in einem Convolutional Neural Network (CNN) entsteht durch folgende Schritte:

- Convolutional Filter** — Kleiner Filter (z. B. 2×2) wird auf Bildbereiche angewendet. Ergebnis wird durch eine Aktivierungsfunktion verarbeitet (z. B. ReLU)
- Pooling Layer** — Kleine Pooling-Matrix (z. B. 2×2) mit Aggregationsfunktion, typischerweise **MAX**

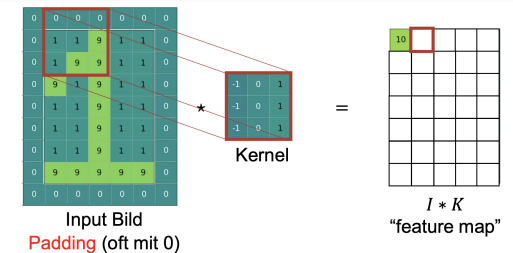
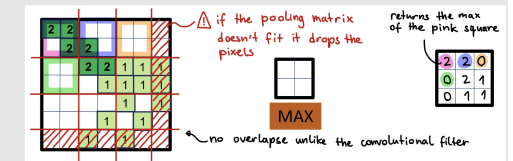
Prozess:

- Anwenden des Filters auf die Eingabematrix
- Anwenden der ReLU-Funktion auf jedes Element des Ergebnisses (setzt jeden negativen Wert auf 0)
- Abschließend: Anwenden der Pooling-Matrix (z. B. MaxPooling)

applying convolutional filter:



applying pulling matrix:



11. Unsupervised Learning

11.1. Clustering

K-Means

K-Means ist ein Unsupervised-Learning-Algorithmus, der n Datenpunkte in K disjunkte Cluster aufteilt, indem die Varianz innerhalb der Cluster minimiert wird. (→ 8.5)

Loss Function (Zielfunktion)

Sei:

- $X = \mathbf{x}_1, \dots, \mathbf{x}_n$ — mit $\mathbf{x}_i \in \mathbb{R}^d$
- μ_k — Centroid des Clusters k
- C_k — Menge der Punkte im Cluster k

Die Verlustfunktion (Distortion/Inertia) lautet:

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \mu_k\|^2$$

Ziel:

$$\min_{C_1, \dots, C_K, \mu_1, \dots, \mu_K} J$$

Lloyd's Algorithm (Standard K-Means)

1. Initialisierung

Wähle K initiale Centroide $\mu_1^{(0)}, \dots, \mu_K^{(0)}$

2. Zuweisungs-Schritt

Weise jedem Punkt seinen nächstgelegenen Centroid zu: $c_i = \arg \min_k \|\mathbf{x}_i - \mu_k\|^2$

3. Update-Schritt

Berechne die Centroide neu als Mittelwert der zugewiesenen Punkte: $\mu_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$

4. Wiederhole

die Schritte 2–3, bis sich die Zuweisungen nicht mehr ändern oder J konvergiert.

5. Teste verschiedene initiale Centroide

um lokale Optima zu vermeiden (mehr als 20 unterschiedliche Startzustände).

Elbow-Kriterium

Um K zu wählen, berechne $J(K)$ für verschiedene Werte von K und plote:

- **x-Achse** — K (Anzahl Cluster)
- **y-Achse** — $J(K)$ (Within-Cluster Sum of Squares)

Der Elbow-Punkt ist dort, wo zusätzliche Cluster nur noch marginalen Gewinn bringen — die Kurve macht einen sichtbaren Knick.

11.2. Dimensionsreduktion

Dimensionsreduktion

Dimensionsreduktionsverfahren stellen hochdimensionale Daten in weniger Dimensionen dar, wobei die wesentliche Struktur erhalten bleibt. Dies erleichtert Visualisierung, Rauschunterdrückung und beschleunigt Lernverfahren.

Principal Component Analysis (PCA)

PCA ist ein lineares Verfahren, das neue orthogonale Achsen (Hauptkomponenten) bestimmt, entlang derer die Varianz der Daten maximal ist. Die erste Hauptkomponente erfasst die größte Varianz, die zweite die nächstgrößte, usw. Die Daten werden anschließend in dieses neue Koordinatensystem projiziert, sodass man von d auf $m \ll d$ Dimensionen reduzieren kann, während der größte Teil der Informationsstreuung erhalten bleibt. (→ 8.4)

t-SNE

t-SNE ist ein nichtlineares Verfahren, das vor allem lokale Nachbarschaften im Datensatz erhält. Es betont kleine Distanzen und bildet dadurch ähnliche Punkte nahe beieinander ab, was oft zu klar getrennten Clustern in 2D oder 3D führt. Es eignet sich besonders für die Visualisierung komplexer Strukturen.

11.3. Zusammenfassung

- **MLP** — Gut für strukturierte Daten (z. B. Tabellen) (→ 10.1)
- **CNN** — Gut für unstrukturierte Bilddaten (→ 10.3)
- **Versteckte Schichten** — meist ReLU
- **Ausgabeschicht:**
 - Sigmoid bei binärer Klassifikation
 - Softmax bei Mehrklassenklassifikation (→ 10.2)
- Lernt erfolgt über Anpassung der Gewichtungen w und Bias-Terme b

12. Code-Beispiele

12.1. Sortieralgorithmen

Quick Sort (Median-of-Three Pivot)

```
def quicksort_median3_pivot(A, left, right):
    if left < right:
        mid = (left + right) // 2
        if A[mid] < A[left]:
            A[left], A[mid] = A[mid], A[left]
        if A[right] < A[left]:
            A[left], A[right] = A[right], A[left]
        if A[right] < A[mid]:
            A[mid], A[right] = A[right], A[mid]

        pivot = A[mid]
        i = left
        j = right - 1
        while i < j:
            while i <= j and A[i] <= pivot:
                i += 1
            while i <= j and A[j] >= pivot:
                j -= 1
            if i < j:
                A[i], A[j] = A[j], A[i]
        A[left], A[j] = A[j], A[left]
        quicksort_median3_pivot(A, left, j)
        quicksort_median3_pivot(A, j + 1, right)
```

Merge Sort

```
def merge_sort(a):
    if len(a) == 1:
        return a
    else:
        s1 = merge_sort(a[:len(a)//2])
        s2 = merge_sort(a[len(a)//2:])
        s = merge(s1, s2)
        return s
def merge(a1, a2):
    b, i, j = [], 0, 0
    while i < len(a1) and j < len(a2):
        if a1[i] < a2[j]:
            b.append(a1[i])
            i += 1
        else:
            b.append(a2[j])
            j += 1
    b += a1[i:]
    b += a2[j:]
    return b
```

Insertion Sort

```
def insertionSort(A, l, r):
    for i in range(l + 1, r):
        j = i - 1
        while j >= 0 and A[j] > A[j + 1]:
            A[j], A[j + 1] = A[j + 1], A[j]
            j -= 1
```

Bubble Sort

```
def bubbleSort(A, l, r):
    for i in range(l, r):
        for j in range(r - i - 1):
            if A[j] > A[j + 1]:
                swap(A, j, j + 1)
```

Selection Sort

```
def selectionSort(A, l, r):
    for i in range(l, r):
        minJ = i
        for j in range(i + 1, r):
            if A[j] < A[minJ]:
                minJ = j
        if minJ != i:
            A[i], A[minJ] = A[minJ], A[i]
```

Binary Insertion Sort

```
# This function will return the index of element
# just greater than 'key' in Array from 0-N
def binarySearch(Array, N, key):
    L = 0
    R = N
    while (L < R):
        mid = (L + R) // 2
        if (Array[mid] <= key):
            L = mid + 1
        else:
            R = mid
    return L

def binaryInsertionSort(Array):
    # We assume the 1st element of Array
    # to be already sorted.
    # Start iterating from the 2nd element
    # to the last element.
    for i in range(1, len(Array)):
        key = Array[i]
        pos = binarySearch(Array, i, key)
        # 'pos' will now contain the index
        # where 'key' should be inserted.
        j = i
        # Shifting every element from 'pos'
        # to 'i' towards right
        while (j > pos):
            Array[j] = Array[j - 1]
            j -= 1
        # Inserting 'key' in its correct position.
        Array[pos] = key
        print("The array after", i, "iterations >=", *Array)
```

Heap Sort

```
def heap_sort(list):
    # turns list into valid heap
    heapify(list)

    for i in reversed(range(len(list))):
        list[0], list[i] = list[i], list[0]
        siftDown(list, 0, i)

    return
```

12.2. Dynamische Programmierung und Memo

Karotten in $n \times n$ Matrix: Bester Weg

```
def longest_path_dp(grid):
    n, m = len(grid), len(grid[0])
    s = [[None] * m for _ in range(n)]
    decision = [[None] * m for _ in range(n)]

    # fill out the DP table:
    for i in range(n - 1, -1, -1):
        for j in range(m - 1, -1, -1):
            if i == n - 1 and j == m - 1:
                s[i][j] = grid[i][j]
                decision[i][j] = "Stop"
            elif i == n - 1 and j < m - 1:
                s[i][j] = grid[i][j] + s[i][j + 1]
                decision[i][j] = "East"
            elif i < n - 1 and j == m - 1:
                s[i][j] = grid[i][j] + s[i + 1][j]
                decision[i][j] = "South"
            else:
                east = grid[i][j] + s[i][j + 1]
                south = grid[i][j] + s[i + 1][j]
                s[i][j] = max(east, south)
                decision[i][j] = "East" if east >= south
                else "South"

    # reconstruct the best path:
    path, i, j = [], 0, 0
    while (i, j) != (n - 1, m - 1):
        path.append(decision[i][j])
        if decision[i][j] == "East":
            j += 1
        else:
            i += 1

    return s[0][0], path
```

Treppensteigen (DP) — Anzahl Wege

```
# How many distinct ways are there to climb
# n number of stairs if you take one or two steps at a time
def climb_stairs(n: int) -> int:
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b
```

Golomb-Zahlen (nur Rekursion)

```
def golomb(n):  
    """  
    pre: integer n > 0  
    post: return the nth Golomb number  
    """  
    if n == 1:  
        return 1  
    else:  
        return 1 + golomb(n=golomb(golomb(n - 1)))
```

Stab schneiden (Rod Cutting)

```
# w: Liste mit Werten für Stabstückchenlänge  
def cuts(n, w):  
    s = [None] * (n + 1)  
    L = [None] * (n + 1)  
  
    for i in range(n + 1):  
        if i == 0:  
            s[i] = 0  
        else:  
            option = 1  
            value = w[1] + s[i - 1]  
            for j in range(1, i + 1):  
                new_option = j  
                new_value = w[j] + s[i - j]  
                if new_value > value:  
                    option = j  
                    value = new_value  
            s[i] = value  
            L[i] = option  
  
    i = n  
    cut = []  
    while i >= 1:  
        cut.append(L[i])  
        i -= L[i]  
  
    return (s[n], cut)
```

12.3. ML / Cross-Validation mit GridSearchCV

Imports

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import warnings  
from sklearn.datasets import make_circles  
from sklearn.model_selection import (train_test_split,  
                                     GridSearchCV)  
from sklearn.neural_network import MLPClassifier  
from sklearn.linear_model import LogisticRegression  
from sklearn.tree import DecisionTreeClassifier, plot_tree  
from sklearn.metrics import accuracy_score  
from utils import plot_decision_boundary, plot_data
```

GridSearchCV — Anwendung

```
# read in the data  
df = pd.read_csv("data.csv")  
X = df[["size", "bright"]].to_numpy()  
y = df["label"].to_numpy()  
  
# split the data into training and testing  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.5, random_state=42)  
  
# Crossvalidate the different tree lengths  
# using the train data  
dt = DecisionTreeClassifier()  
# tree depth from 1 to 9  
grid = {"max_depth": range(1, 10)}  
  
grid = GridSearchCV(dt, param_grid=grid, cv=5)  
grid.fit(X_train, y_train)  
  
best_dt = grid.best_estimator_  
  
y_pred_train = best_dt.predict(X_train)  
y_pred_test = best_dt.predict(X_test)  
  
acc_train = accuracy_score(y_train, y_pred_train)  
acc_test = accuracy_score(y_test, y_pred_test)  
  
print("Accuracy on train data: {:.2f}".format(acc_train))  
print("Accuracy on test data: {:.2f}".format(acc_test))
```

Knapsack DP

```
def knapsack(weights, values, capacity):  
    n = len(weights)  
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]  
  
    for i in range(1, n + 1):  
        for w in range(capacity + 1):  
            if weights[i-1] <= w:  
                dp[i][w] = max(  
                    dp[i-1][w],  
                    dp[i-1][w - weights[i-1]] + values[i-1]  
                )  
            else:  
                dp[i][w] = dp[i-1][w]  
  
    return dp[n][capacity]
```

Optimal Binary Search Tree DP

```
def optimal_bst(freq):  
    n = len(freq)  
    dp = [[0]*n for _ in range(n)]  
    sum_freq = [[0]*n for _ in range(n)]  
  
    for i in range(n):  
        sum_freq[i][i] = freq[i]  
        for j in range(i+1, n):  
            sum_freq[i][j] = sum_freq[i][j-1] + freq[j]  
  
    for length in range(1, n+1):  
        for i in range(n - length + 1):  
            j = i + length - 1  
            dp[i][j] = float('inf')  
            for r in range(i, j+1):  
                cost = (dp[i][r-1] if r > i else 0) + \  
                    (dp[r+1][j] if r < j else 0) + \  
                    sum_freq[i][j]  
                dp[i][j] = min(dp[i][j], cost)  
  
    return dp[0][n-1]
```

Graph DP (Memoization)

```
def graph_dp(node, graph, dp):
    if node in dp:
        return dp[node]

    if not graph[node]: # leaf
        dp[node] = 0
        return 0

    max_cost = 0
    for neighbor, weight in graph[node]:
        cost = weight + graph_dp(neighbor, graph, dp)
        max_cost = max(max_cost, cost)

    dp[node] = max_cost
    return max_cost
```

Longest Common Subsequence (LCS) — memoisiert

```
def lcs(seq1, seq2):
    memo = {}

    def dp(i, j):
        if i == len(seq1) or j == len(seq2):
            return 0
        if (i, j) in memo:
            return memo[(i, j)]
        if seq1[i] == seq2[j]:
            memo[(i, j)] = 1 + dp(i + 1, j + 1)
        else:
            memo[(i, j)] = max(
                dp(i + 1, j),
                dp(i, j + 1)
            )
        return memo[(i, j)]

    return dp(0, 0)
```

Springer-Matrix
mit Recursion

```
def rec_best_way(a, i = 0, j = 0):
    n, m = len(a), len(a[0])

    reward1, reward2, reward3, reward4 = 0, 0, 0, 0
    if i - 2 >= 0 and j + 1 < m:
        reward1 = rec_best_way(a, i - 2, j + 1)
    if i - 1 >= 0 and j + 2 < m:
        reward2 = rec_best_way(a, i - 1, j + 2)
    if i + 1 < n and j + 2 < m:
        reward3 = rec_best_way(a, i + 1, j + 2)
    if i + 2 < n and j + 1 < m:
        reward4 = rec_best_way(a, i + 2, j + 1)
    return a[i][j] + max([reward1, reward2, reward3, reward4])
```

mit Dynamic Programming:

```
def dp_best_way(a):
    n, m = len(a), len(a[0])

    s = [[None for _ in range(m)] for _ in range(n)]

    for j in range(m - 1, -1, -1):
        for i in range(n - 1, -1, -1):
            reward1, reward2, reward3, reward4 = 0, 0, 0, 0
            if i - 2 >= 0 and j + 1 < m:
                reward1 = s[i - 2][j + 1]
            if i - 1 >= 0 and j + 2 < m:
                reward2 = s[i - 1][j + 2]
            if i + 1 < n and j + 2 < m:
                reward3 = s[i + 1][j + 2]
            if i + 2 < n and j + 1 < m:
                reward4 = s[i + 2][j + 1]
            s[i][j] = a[i][j] + max([reward1, reward2, reward3, reward4])

    return s[0][0]
```

Beispiel Gini Index Berechnung

Bereich 1: $R = 2, B = 0$
 $p_r = 1.0, p_b = 0.0$
 $G_1 = 1 - \left(\frac{\text{Rote Punkte im Bereich}}{\text{Alle Punkte im Bereich}} \right)^2 - \left(\frac{\text{Blaue Punkte}}{\text{Alle Punkte im Bereich}} \right)^2$
 $G_1 = 1 - (1.0^2 + 0.0^2) = 0.00$

Bereich 3: $R = 1, B = 4$
 $p_r = \frac{1}{5} = 0.2, p_b = 0.8$
 $G_3 = 1 - (0.2^2 + 0.8^2) = 1 - (0.04 + 0.64) = 0.32$

Bereich 2: $R = 3, B = 1$
 $p_r = \frac{3}{4} = 0.75, p_b = 0.25$
 $G_2 = 1 - (0.75^2 + 0.25^2) = 1 - (0.5625 + 0.0625) = 0.375$

Bereich 4: $R = 3, B = 1$
 $p_r = 0.75, p_b = 0.25$
 $G_4 = 1 - (0.75^2 + 0.25^2) = 0.375$

Gesamtanzahl: $n_1 = 2, n_2 = 4, n_3 = 5, n_4 = 4, N = 15$

$$G_{\text{gesamt}} = \frac{n_1}{N} G_1 + \frac{n_2}{N} G_2 + \frac{n_3}{N} G_3 + \frac{n_4}{N} G_4$$
$$= \frac{2}{15} (0.00) + \frac{4}{15} (0.375) + \frac{5}{15} (0.32) + \frac{4}{15} (0.375)$$
$$= 0 + 0.1000 + 0.1067 + 0.1000 \approx 0.3067$$

$G_{\text{gesamt}} \approx 0.31$

DP für Linearkombinationen

```
def possible_amounts(max_amount, denominations):
    """example : possible_amounts(20, [5, 2]) returns [0, 2, 4,
    5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
    all linear combinations up to 20 of 5 and 2.
    """
    n = max_amount
    s = [False] * (n + 1)
    s[0] = True

    for i in range(1, n+1):
        s[i] = False
        for d in denominations:
            if i - d >= 0 and s[i - d]:
                s[i] = True
    return s
```

12.4. Linked Lists

```
def mergeTwoLists(self, list1: Optional[ListNode], list2:
Optional[ListNode]) ->Optional[ListNode]:
    # Create a dummy node to start the new list
    dummy = ListNode(0)
    # This will always point to the last node
    last = dummy
    #in the new list
    # Pointer for traversing list1
    p1 = list1
    # Pointer for traversing list2
    p2 = list2

    # Traverse both lists and merge them
    # by comparing node values
    while p1 is not None and p2 is not None:
        if p1.val <= p2.val:
            last.next = p1
            p1 = p1.next
        else:
            last.next = p2
            p2 = p2.next
        last = last.next

    # Attach the remaining nodes of the non-empty list
    if p1 is not None:
        last.next = p1
    else:
        last.next = p2

    return dummy.next
# The first node is dummy, so return the next node
```

12.5. String umkehren

```
#reverses a string cut into intervals: ABCDE
#with interval 2 becomes BADCE
def reverse_string(text, interval):
    reverse = ""
    for i in range(0, len(text), interval):
        reverse += text[i:i+interval:1][::-1]
    return reverse
```

13. DP-Quiz

13.1. Welche Aussage ist korrekt?

Welche der folgenden Aussagen ist wahr?

- c. Dynamische Programmierung eignet sich am besten für Probleme mit optimaler Substruktur und überlappenden Teilproblemen.

Welche der folgenden Aussagen über dynamische Programmierung sind wahr?

- (I) Dynamische Programmierung ist nur anwendbar, wenn man mit einem 2D-Array arbeitet oder eine 2D-DP-Tabelle aufbaut.
(II) Dynamische Programmierung benötigt mindestens zwei verschachtelte for-Schleifen.
• d. Keine der Aussagen ist wahr.

Welche der folgenden Aussagen über dynamische Programmierung sind wahr?

- (I) Wenn ein Algorithmus eine zweidimensionale Matrix befüllt, dann handelt es sich um dynamische Programmierung.
(II) Wenn ein Algorithmus verschachtelte for-Schleifen benutzt, dann ist es dynamische Programmierung.
• d. Keine der Aussagen ist wahr.

Du und dein Kollege haben unabhängig voneinander DP-Lösungen für dasselbe Problem entwickelt. Beim Vergleich stellst du fest, dass dein Kollege andere Teilprobleme und eine DP-Tabelle mit mehr Dimensionen verwendet hat. Was beschreibt die Korrektheit eurer Lösungen am besten?

- a. Es ist möglich, dass beide korrekte, valide und effiziente DP-Lösungen haben.

Welche der folgenden Aussagen über dynamische Programmierung und Optimierungsprobleme sind immer wahr?

- (I) Dynamische Programmierung befasst sich mit Optimierungsproblemen.
(II) Wenn ein Problem ein Optimierungsproblem ist, kann dynamische Programmierung zur Lösung verwendet werden.
• b. Nur (I) ist wahr.

Die n -te Fibonacci-Zahl $F(n)$ ist definiert als $F(n) = F(n - 1) + F(n - 2)$ mit den Basisfällen $F(1) = 1$ und $F(2) = 2$. Betrachte einen **Top-down**-DP-Algorithmus, der n als Eingabe nimmt und die n -te Fibonacci-Zahl berechnet. Was beschreibt die Notwendigkeit von Memoization am besten?

- b. Memoization ist notwendig, weil Werte (also kleinere Fibonacci-Zahlen) sonst mehrfach berechnet werden müssten.
• d. Memoization ist notwendig, weil unsere DP von einer festen, endlichen Anzahl von Werten abhängt.

Gegeben sei folgendes Problem: Gegeben ein Integer-Array, gib die Anzahl der längsten streng monoton steigenden Teilfolgen zurück. Welche der folgenden Aussagen ist FALSCH?

- a. Eine effiziente algorithmische Lösung dieses Problems kann nicht als DP kategorisiert werden, weil sie keine Min/Max-Optimierung enthält.

13.2. Wahr oder falsch

Welche der folgenden Aussagen über Basisfälle in einem DP-Algorithmus sind korrekt? Mehrfachauswahl.

- a. Basisfälle sind immer notwendig. **Wahr**
- b. Die Korrektheit der durch die Rekurrenz berechneten Werte für die überlappenden Teilprobleme hängt von der Korrektheit der Basisfälle ab. **Wahr**
- c. Basisfälle werden immer einem Wert zugewiesen, ohne Berechnungen oder Vergleiche durchzuführen. **Falsch**
- d. Die Anzahl der Basisfälle muss nie 3 überschreiten. **Falsch**

Dynamische Programmierung erlaubt es uns immer sofort, einen Teil der Lösung mit einem einzigen Durchlauf der Eingabe zu bestimmen.

- b. Falsch

Welche der folgenden Aussagen zur Rückgabe von Lösungen in einem DP-Algorithmus sind immer wahr? Mehrfachauswahl.

- a. Sobald eine DP-Tabelle korrekt befüllt ist, genügt ein einziger Lookup, um die optimale Lösung zu bestimmen. **Wahr**
- b. Sobald eine DP-Tabelle korrekt befüllt ist, steht der Wert der optimalen Lösung in einem der Tabelleneinträge oder ist effizient aus den Tabellenwerten berechenbar. **Wahr**
- c. Sobald eine DP-Tabelle korrekt befüllt ist, kann der Wert der optimalen Lösung an mehreren Stellen in der Tabelle stehen. **Wahr**

Welche der folgenden Anforderungen gelten für die Teilprobleme eines DP-Algorithmus? Mehrfachauswahl.

- a. Die Teilprobleme müssen Memoization verwenden, sonst ist keine Lösung möglich. **Falsch**
- b. Die Teilprobleme müssen sich einem Basisfall annähern, sonst ist keine Lösung möglich. **Wahr**
- c. Jedes Teilproblem muss direkt einen Basisfall aufrufen, sonst ist keine Lösung möglich. **Falsch**
- d. Die Teilprobleme müssen alle unterschiedliche Werte für die relevanten Variablen verwenden, sonst ist keine Lösung möglich. **Falsch**

Angenommen wir haben einen rekursiven DP-Algorithmus, verzichten aber auf Memoization. Welche Probleme können auftreten? Mehrfachauswahl.

- a. Der Algorithmus speichert Informationen, die nicht notwendig sind, weil sie später nicht verwendet werden. **Wahr**
- b. Der Algorithmus ist inkorrekt und gibt die falsche Antwort zurück. **Falsch**
- c. Der Algorithmus ist ineffizient und benötigt deutlich mehr Zeit als nötig. **Wahr**
- d. Der Algorithmus löst die Basisfälle nicht korrekt. **Falsch**
- e. Der Algorithmus löst dieselben Teilprobleme mehrfach. **Wahr**
- f. Der Algorithmus verwendet nicht die korrekte Rekurrenz. **Falsch**

Welche Schritte sind beim Entwurf eines korrekten und effizienten DP-Algorithmus immer notwendig? Mehrfachauswahl.

- a. Eine Rekurrenz entwerfen, mit der sich der Wert für alle relevanten Teilprobleme berechnen lässt. **Wahr**
- b. In der Implementierung die Lösungen aller Teilprobleme speichern, die mehr als einmal gelöst werden müssten. **Wahr**
- c. Korrekte Basisfälle für die Rekurrenz haben. **Wahr**

Für welche Eingabetypen kann dynamische Programmierung ein möglicher Lösungsansatz sein? Mehrfachauswahl.

- a. Eingabe: ein Integer-Array. **Wahr**
- b. Eingabe: ein einzelner Integer. **Wahr**
- c. Eingabe: eine zweidimensionale Integer-Matrix. **Wahr**
- d. Eingabe: ein ungerichteter Graph mit n Knoten und m Kanten. **Wahr**

Dynamische Programmierung liefert für jedes berechenbare Problem eine Lösung in polynomieller Zeit.

- b. Falsch

13.3. Code-Fragen

Frage: Welche der folgenden Implementierungen liefert die n -te Tribonacci-Zahl mit der geringsten Zeitkomplexität (asymptotische Gleichheit erlaubt)? Mehrfachauswahl.

```
function solve(n, seq):
    if seq[n] != NULL:
        return seq[n]
    if n == 0: return 0
    if n == 1 or n == 2: return 1
    seq[n] = (solve(n-1, seq) + solve(n-2, seq)
              + solve(n-3, seq))
    return seq[n]
```

```
function main(n):
    1D array seq
    return solve(n, seq)
```

```
function main(n):
    if n == 0: return 0
    if n == 1 or n == 2: return 1
    1D array seq
    seq[0] = 0
    seq[1] = 1
    seq[2] = 1
    for i = 3 to n + 1:
        seq[i] = seq[i-1] + seq[i-2] + seq[i-3]
    return seq[n]
```

Die n -te Fibonacci-Zahl $F(n)$ ist definiert als $F(n) = F(n-1) + F(n-2)$ mit den Basisfällen $F(1) = 1$ und $F(2) = 2$. Dein Kollege schlägt folgende Lösung zur Berechnung der n -ten Fibonacci-Zahl vor:

```
function Fib(n)
    if n == 1: return 1
    else if n == 2: return 1
    else: return Fib(n - 1) + Fib(n - 2)
```

Ist die Antwort korrekt und effizient?

- b. Die Lösung ist korrekt, aber nicht effizient, weil sie nicht memoisiert und dadurch viele Funktionsaufrufe wiederholt.

Gegeben sei folgendes Problem: Wir möchten die Wahrscheinlichkeit kennen, dass eine bestimmte Anzahl Regentage in den nächsten n Tagen erreicht wird. Für jeden Tag i ist $p_i \in [0,1]$ die Regenwahrscheinlichkeit. Gegeben sei zudem ein Schwellwert k . Gib die Wahrscheinlichkeit zurück, dass es an mindestens k Tagen regnet. **Frage:** Du darfst annehmen, dass (a) und (b) beide die korrekte Ausgabe liefern. Welche der beiden ist eine DP-Lösung?

```
function ChanceOfRain(p[], k)
    2D array memo
    return ChanceOfRainH(p[], 0, k, memo)
```

```
function ChanceOfRainH(p[], i, k, memo[][]):
    if k == 0:
        return 1
    if i == len(p):
        return 0
    if memo[i][k] != NULL:
        return memo[i][k]
    Rain = p[i] * ChanceOfRainH(p[], i + 1, k - 1, memo)
    Sunny = (1 - p[i]) * ChanceOfRainH(p[], i + 1, k, memo)
    Memo[i][k] = Rain + Sunny
    return Memo[i][k]
```

```
function ChanceOfRain(p[], k)
    2D array table
    for i == 0 to k:
        table[n][i] = 0
    for i = n to 1:
        table[i][0] = 1
    for i = n to 1:
        for j = 1 to k:
            rain = table[i+1][j-1] * p[i]
            sunny = table[i+1][j] * (1 - p[i])
            table[i][j] = rain + sunny
    return table[0][k]
```

Gegeben sei folgendes Problem: Du möchtest möglichst viel Zeit mit Lesen von Büchern aus deinem Bücherregal verbringen. Du darfst aber kein Buch lesen, das direkt neben einem bereits gelesenen Buch steht. Gegeben ist ein Array mit den Lesezeiten der Bücher. Gib einen Algorithmus an, der die maximale Lesezeit zurückgibt.

Frage: Hier ist ein Lösungsversuch:

```
function BookRead(books[], i, partialTimes[])
    if i >= len(books):
        return 0
    if partialTimes[i] != NULL:
        return partialTimes[i]
    take = BookRead(books, i + 2, partialTimes) + books[i]
    leave = BookRead(books, i + 1, partialTimes)
    return max(take, leave)
```

Was verhindert, dass diese Lösung eine DP-Lösung ist?

- c. Das Array `partialTimes` wird nicht korrekt verwendet. **(korrekt)**

Das Coin-Row-Problem ist wie folgt definiert: Gegeben ist eine Reihe von n Münzen mit positiven Werten $c_1, c_2, \dots, c_n$. Ziel ist es, Münzen mit maximalem Gesamtwert so auszuwählen, dass keine zwei in der Reihe direkt benachbarten Münzen gemeinsam genommen werden.

Sei das Teilproblem $D[i]$ die Lösung mit maximalem Wert über die ersten i Münzen:

$$D[i] = \max(c_i + D[i - 2], D[i - 1])$$

Beispiel:

Eingabe: [7, 15, 12] → Lösung: 7 + 12 = 19

Eingabe: [10, 4, 4, 10, 4, 4, 10] → Lösung: 10 + 10 + 10 = 30

Lösungsversuch (DP):

```
1D Array partialSums
function coinRow(coins, n):
# coins is an array of coin values length n
if n == 1:
    partialSums[n] = coins[0]
else if n == 2:
    partialSums[n] = max(coins[0], coins[1])
else:
    partialSums[n] = max(coins[n] + coinRow(coins, n - 2),
        coinRow(coins, n - 1))
return partialSums[n]
```

Frage: Ist der Code korrekt?

- b. Der Code verwendet Memoization nicht korrekt, daher ist die Laufzeit exponentiell.

Gegeben ein positiver Integer K . Bestimme die minimale Anzahl Operationen, um 0 in K zu überführen. Erlaubte Operationen:

- Eins zum Operanden addieren.
- Operanden mit 2 multiplizieren.

Betrachte diese gültige DP-Lösung:

```
function min_operation(k):
    1D Array arr # array of size k+1
    arr[0] = 0
    for i = 1 TO k:
        arr[i] = arr[i - 1] + 1
        if i % 2 == 0:
            arr[i] = min(arr[i], arr[floor(i / 2)] + 1)
    return arr[k]
```

Welche der folgenden Aussagen ist wahr?

- c. Die Lösung ist ein gültiger eindimensionaler DP-Ansatz.

Gegeben ist ein Gitter der Grösse $n \times m$, wobei jede Zelle einen nicht-negativen Integer enthält, der die Kosten für das Betreten der Zelle angibt. Startpunkt ist die linke untere Ecke $(n-1, 0)$, Ziel ist die rechte obere Ecke $(0, m-1)$. Minimiere die Gesamtkosten. Lösung:

```
function minCostPathToTopRight(grid[][]):
    n = len(grid) # number of rows
    m = len(grid[0]) # number of columns

    # Initialize a 2D DP array to store minimum cost values
    # all elements initialized to 0
    2D Array partialCosts

    # Fill in the DP array
    for i = n - 1 TO 0:
        for j = 0 TO m:
            # Initialize the bottom-left cell
            if i == n - 1 and j == 0:
                partialCosts[i][j] = grid[i][j]
            else if i == n - 1:
                partialCosts[i][j] = grid[i][j] +
                    partialCosts[i][j - 1]
            else if j == 0:
                partialCosts[i][j] = grid[i][j] +
                    partialCosts[i + 1][j]
            else:
                partialCosts[i][j] = grid[i][j] +
                    min(partialCosts[i][j - 1],
                        partialCosts[i + 1][j])

    return partialCosts[0][m - 1]
```

Wähle die korrekte Aussage:

- b. Dies ist eine gültige DP-Lösung, die das Gitter von links unten nach rechts oben durchläuft.

Das Stock-Market-Problem ist wie folgt definiert: Gegeben eine Sequenz historischer Tageskurse einer Aktie über n aufeinanderfolgende Tage. Bestimme die Tage, an denen man die Aktie hätte kaufen und verkaufen sollen, um den Gewinn zu maximieren, und gib den maximalen Gewinn zurück. Hinweis: Es darf nur einmal gekauft und einmal verkauft werden, und der Kauftag muss vor oder gleich dem Verkaufstag liegen.

Beispiele:

Eingabe: [2, 5, 7, 1] → Lösung: 5 (Kauf an Tag 1, Verkauf an Tag 3)

Eingabe: [5, 2, 4, 8, 5, 6, 7] → Lösung: 6 (Kauf an Tag 2, Verkauf an Tag 4)

Welche der folgenden Funktionen ist eine DP-Lösung des Stock-Market-Problems?

```
function maxProfit(prices[]):
    minPrice = min(prices)
    minIndex = len(prices)
    maxPrice = 0
    for i = 1 to len(prices):
        if prices[i] == minPrice:
            minIndex = i
        if minIndex < i and prices[i] > maxPrice:
            maxPrice = prices[i]
    return max(0, maxPrice - minPrice)
```

```
function maxProfit(prices[]):
    if (len(prices) == 0):
        return 0
    maxProfit = 0
    minPrice = prices[0]
    for i = 1 to len(prices):
        if prices[i] < minPrice:
            minPrice = prices[i]
        else:
            maxProfit = max(maxProfit, prices[i] - minPrice)
    return maxProfit
```

Das Longest-Common-Subsequence-Problem (LCS) ist wie folgt definiert: Gegeben zwei Strings (Zeichen-Arrays) $X[0..m-1]$ und $Y[0..n-1]$ der Länge m und n . Finde die Länge der längsten Teilfolge, die in X und Y vorkommt. Beispiel:

$X = [a, b, c, d, e, f]$

$Y = [d, z, a, f, c, w, d, e]$

Antwort = 4 (Teilfolge [a, c, d, e])

Welche der folgenden Funktionen löst das LCS-Problem mit der geringsten Zeitkomplexität? Eine Antwort.

```
# arr is of appropriate size
function lcs(X, Y, m, n, arr[]):
    if (m == 0 or n == 0):
        return 0
    if (arr[m][n] != NULL):
        return arr[m][n]
    if X[m - 1] == Y[n - 1]:
        arr[m][n] = 1 + lcs(X, Y, m - 1, n - 1, arr)
        return arr[m][n]
    arr[m][n] = max(lcs(X, Y, m, n - 1, arr), lcs(X, Y,
        m - 1, n, arr))
    return arr[m][n]
```

```
# arr is of appropriate size
function lcs(X, Y, m, n, arr[]):
    for i = 0 to m:
        for j = 0 to n:
            if i == 0 or j == 0:
                arr[i][j] = 0
            else if X[i - 1] == Y[j - 1]:
                arr[i][j] = arr[i - 1][j - 1] + 1
            else:
                arr[i][j] = max(arr[i - 1][j], arr[i][j - 1])
    return arr[m][n]
```

```

# print(dir(math)) falls syntax unbekannt
import math
c=0
def f(): global c; c+=1
def check(g):
    S=[10,14,18,22]; C=[]
    for n in S: global c; c=0; g(n); C.append(c)
    T={"log n":math.log2, "sqrt(n)":lambda n:n**.5}
    E={}
    for k,h in T.items():
        try:
            R=[C[i]/h(n) for i,n in enumerate(S) if h(n)>1e-9]
            if not R: continue
            M=sum(R)/len(R)
            if M<1e-9: continue
            E[k]=math.sqrt(sum((x-M)**2 for x in R)/len(R))/M
        except: continue
    return "theta("+min(E,key=E.get)+")" if E else "theta(?)"

def g(n): # g von Aufgabe einfüegen
    pass

print(check(g)) # Ergebnis printen

```

14. Mitwirkende

HINWEIS ZUM DRUCKEN

Um die Zusammenfassung drucken zu können, muss die sogenannte "Double-Printing"-Technik angewendet werden. Zwei Seiten werden somit auf eine Seite gedruckt, was es ermöglicht, mehr in die Prüfung mitzunehmen.

Diese Zusammenfassung wurde von Georg Niggli und Nina Burren für die Vorlesung Informatik II von Ralf Sasse und Carlos Cotrini (Frühlingssemester 2025) erstellt. Für die Codebeispiele wurde eine CodeExpert-Vorlage von Overleaf erheblich umgestaltet.

Es ist jedem freigestellt, diese Zusammenfassung weiterzuentwickeln und zu veröffentlichen. Um Verwirrung zu vermeiden, sollte jedoch deutlich darauf hingewiesen werden, dass es sich nicht um die Originalfassung handelt und die Credits erhalten bleiben müssen.

Richtigkeit und Vollständigkeit können nicht garantiert werden. Die $\text{\LaTeX}$ -Datei kann auf Anfrage bereitgestellt werden: Bitte verwende dafür den Betreff *INFORMATIK II - LATEX*
gniggli@student.ethz.ch oder georg.niggli@aris-space.ch